milestone_3

April 17, 2026

Milestone 3: Neural Network Designs

[1]:
```
import tensorflow as tf
from sklearn.pipeline import Pipeline
from scikeras.wrappers import KerasClassifier
from pathlib import Path
from time import strftime
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,
                                                     restore_best_weights=True)

def get_run_logdir(root_logdir='my_logs'):
    return Path(root_logdir)/strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir()

tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir,
                                                profile_batch=(100, 200))


def create_baseline(meta):
    X_shape_ = meta["X_shape_"]
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(300, activation="relu",␣
 ↪input_shape=(X_shape_[1],)),
        tf.keras.layers.Dense(300, activation="relu"),
        tf.keras.layers.Dense(4, activation="softmax")
    ])
    model.compile(optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy'])
    return model
```

```
c:\Users\samue\anaconda3\Lib\site-packages\pandas\core\arrays\masked.py:56:
UserWarning: Pandas requires version '1.4.2' or newer of 'bottleneck' (version
'1.3.7' currently installed).
  from pandas.core import (
```

[2]:
```
clf = KerasClassifier(model=create_baseline, epochs=50, batch_size=32,␣
↪verbose=1)
```

The results array will hold all of our results for each model to see which one performed the best for different metrics in the end

[3]:
```
results = []
```

[4]:
```
import numpy as np
from sklearn.svm import LinearSVC
data = np.load('octmnist.npz')
print("Keys in octmnist.npz: ", data.files)

for k in data.files:
    arr = data[k]
    print(f"{k:>12}: shape = {arr.shape}")
```

```
Keys in octmnist.npz:  ['train_images', 'val_images', 'test_images',
'train_labels', 'val_labels', 'test_labels']
train_images: shape = (97477, 28, 28)
  val_images: shape = (10832, 28, 28)
 test_images: shape = (1000, 28, 28)
train_labels: shape = (97477, 1)
  val_labels: shape = (10832, 1)
 test_labels: shape = (1000, 1)
```

[5]:
```
train_images = data["train_images"]
train_labels = data["train_labels"].ravel()

val_images = data["val_images"]
val_labels = data["val_labels"].ravel()

test_images = data["test_images"]
test_labels = data["test_labels"].ravel()
```

[6]:
```
from utils import flattener
from sklearn.preprocessing import MinMaxScaler

pipe_baseline = Pipeline([
    ("Flatten", flattener),
    ("Scale", MinMaxScaler()),
    ("model", clf)
])
```

[7]:
```
from utils import train_eval_model
from utils import stratified_subset

x_tr, y_tr = stratified_subset(train_images, train_labels, 22000)
```

```
x_va, y_va = stratified_subset(val_images, val_labels, 2000)
```

[8]:
```
from sklearn.utils.class_weight import compute_sample_weight

sample_weights = compute_sample_weight(class_weight="balanced", y=y_tr)
sample_weights_valid = compute_sample_weight(class_weight="balanced", y=y_va)
```

[9]:
```
results.append(train_eval_model(
    name="Baseline ANN",
    model=pipe_baseline,
    x_tr=x_tr,
    y_tr=y_tr,
    x_va=x_va,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb, tensorboard_cb]
))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
Please use WSL2 or the TensorFlow-DirectML plugin.

c:\Users\samue\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/50
688/688             3s 2ms/step -
accuracy: 0.4031 - loss: 1.3165
Epoch 2/50
688/688             2s 2ms/step -
accuracy: 0.4885 - loss: 1.2081
Epoch 3/50
688/688             2s 2ms/step -
accuracy: 0.5121 - loss: 1.1541
Epoch 4/50
688/688             2s 2ms/step -
accuracy: 0.5374 - loss: 1.1000
Epoch 5/50
688/688             2s 2ms/step -
accuracy: 0.5459 - loss: 1.0755
Epoch 6/50
688/688             2s 2ms/step -
accuracy: 0.5675 - loss: 1.0360
Epoch 7/50
688/688             2s 2ms/step -
accuracy: 0.5717 - loss: 1.0122
```

```
Epoch 8/50
688/688             2s 2ms/step -
accuracy: 0.5831 - loss: 0.9847
Epoch 9/50
688/688             2s 2ms/step -
accuracy: 0.5924 - loss: 0.9569
Epoch 10/50
688/688             2s 2ms/step -
accuracy: 0.5985 - loss: 0.9445
Epoch 11/50
688/688             2s 2ms/step -
accuracy: 0.6045 - loss: 0.9234
Epoch 12/50
688/688             2s 2ms/step -
accuracy: 0.6118 - loss: 0.9073
Epoch 13/50
688/688             2s 2ms/step -
accuracy: 0.6217 - loss: 0.8917
Epoch 14/50
688/688             2s 2ms/step -
accuracy: 0.6228 - loss: 0.8796
Epoch 15/50
688/688             2s 2ms/step -
accuracy: 0.6315 - loss: 0.8624
Epoch 16/50
688/688             2s 2ms/step -
accuracy: 0.6369 - loss: 0.8490
Epoch 17/50
688/688             2s 2ms/step -
accuracy: 0.6404 - loss: 0.8423
Epoch 18/50
688/688             2s 2ms/step -
accuracy: 0.6417 - loss: 0.8229
Epoch 19/50
688/688             2s 2ms/step -
accuracy: 0.6471 - loss: 0.8192
Epoch 20/50
688/688             2s 2ms/step -
accuracy: 0.6492 - loss: 0.8090
Epoch 21/50
688/688             2s 2ms/step -
accuracy: 0.6546 - loss: 0.7984
Epoch 22/50
688/688             2s 2ms/step -
accuracy: 0.6572 - loss: 0.7893
Epoch 23/50
688/688             2s 2ms/step -
accuracy: 0.6608 - loss: 0.7804
```

```
Epoch 24/50
688/688             2s 2ms/step -
accuracy: 0.6654 - loss: 0.7727
Epoch 25/50
688/688             2s 2ms/step -
accuracy: 0.6685 - loss: 0.7609
Epoch 26/50
688/688             2s 2ms/step -
accuracy: 0.6728 - loss: 0.7527
Epoch 27/50
688/688             2s 2ms/step -
accuracy: 0.6740 - loss: 0.7433
Epoch 28/50
688/688             2s 2ms/step -
accuracy: 0.6775 - loss: 0.7379
Epoch 29/50
688/688             2s 2ms/step -
accuracy: 0.6794 - loss: 0.7314
Epoch 30/50
688/688             2s 2ms/step -
accuracy: 0.6819 - loss: 0.7268
Epoch 31/50
688/688             2s 2ms/step -
accuracy: 0.6826 - loss: 0.7156
Epoch 32/50
688/688             2s 2ms/step -
accuracy: 0.6903 - loss: 0.7039
Epoch 33/50
688/688             2s 2ms/step -
accuracy: 0.6920 - loss: 0.6997
Epoch 34/50
688/688             2s 2ms/step -
accuracy: 0.6906 - loss: 0.6967
Epoch 35/50
688/688             2s 2ms/step -
accuracy: 0.6975 - loss: 0.6852
Epoch 36/50
688/688             2s 2ms/step -
accuracy: 0.6990 - loss: 0.6894
Epoch 37/50
688/688             2s 2ms/step -
accuracy: 0.7004 - loss: 0.6760
Epoch 38/50
688/688             2s 2ms/step -
accuracy: 0.7048 - loss: 0.6706
Epoch 39/50
688/688             2s 2ms/step -
accuracy: 0.7092 - loss: 0.6615
```

```
Epoch 40/50
688/688             2s 2ms/step -
accuracy: 0.7117 - loss: 0.6528
Epoch 41/50
688/688             2s 2ms/step -
accuracy: 0.7124 - loss: 0.6474
Epoch 42/50
688/688             2s 2ms/step -
accuracy: 0.7167 - loss: 0.6424
Epoch 43/50
688/688             2s 2ms/step -
accuracy: 0.7136 - loss: 0.6396
Epoch 44/50
688/688             2s 2ms/step -
accuracy: 0.7180 - loss: 0.6355
Epoch 45/50
688/688             2s 3ms/step -
accuracy: 0.7217 - loss: 0.6249
Epoch 46/50
688/688             2s 2ms/step -
accuracy: 0.7222 - loss: 0.6199
Epoch 47/50
688/688             2s 2ms/step -
accuracy: 0.7260 - loss: 0.6144
Epoch 48/50
688/688             2s 3ms/step -
accuracy: 0.7258 - loss: 0.6122
Epoch 49/50
688/688             2s 2ms/step -
accuracy: 0.7304 - loss: 0.6026
Epoch 50/50
688/688             2s 3ms/step -
accuracy: 0.7335 - loss: 0.5987
63/63             0s 2ms/step
Model: Baseline ANN
Training Time: 83.323
acc: 0.6750
prec:0.7583
recall:0.6750
```

To absolutely no ones suprise even the completely untuned most baseline neural network possible sort of destroyed all other models.

[10]:
```
from utils import median, avg_2x2_pool, flatten_data
from sklearn.preprocessing import FunctionTransformer
```

[11]:
```
preprocess_nn = Pipeline([
    ("median", FunctionTransformer(median)),
    ("avg", FunctionTransformer(avg_2x2_pool)),
    ("flat", FunctionTransformer(flatten_data)),
    ("scale", MinMaxScaler()),
    ("model", clf)
])

results.append(train_eval_model(
    name="avg+median filtering results",
    model=preprocess_nn,
    x_tr=x_tr,
    x_va=x_va,
    y_tr=y_tr,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb, tensorboard_cb]
))
```

```
Epoch 1/50
```

```
c:\Users\samue\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
688/688             2s 2ms/step -
accuracy: 0.4316 - loss: 1.3060
Epoch 2/50
688/688             1s 2ms/step -
accuracy: 0.5071 - loss: 1.2045
Epoch 3/50
688/688             1s 2ms/step -
accuracy: 0.5243 - loss: 1.1510
Epoch 4/50
688/688             1s 2ms/step -
accuracy: 0.5352 - loss: 1.1174
Epoch 5/50
688/688             1s 2ms/step -
accuracy: 0.5441 - loss: 1.0822
Epoch 6/50
688/688             1s 2ms/step -
accuracy: 0.5532 - loss: 1.0655
Epoch 7/50
688/688             1s 2ms/step -
accuracy: 0.5625 - loss: 1.0412
Epoch 8/50
688/688             1s 2ms/step -
```

```
accuracy: 0.5633 - loss: 1.0222
Epoch 9/50
688/688             1s 2ms/step -
accuracy: 0.5760 - loss: 1.0016
Epoch 10/50
688/688             1s 2ms/step -
accuracy: 0.5770 - loss: 0.9929
Epoch 11/50
688/688             1s 2ms/step -
accuracy: 0.5831 - loss: 0.9808
Epoch 12/50
688/688             1s 2ms/step -
accuracy: 0.5782 - loss: 0.9693
Epoch 13/50
688/688             1s 2ms/step -
accuracy: 0.5878 - loss: 0.9490
Epoch 14/50
688/688             1s 2ms/step -
accuracy: 0.5942 - loss: 0.9444
Epoch 15/50
688/688             1s 2ms/step -
accuracy: 0.6008 - loss: 0.9248
Epoch 16/50
688/688             1s 2ms/step -
accuracy: 0.5995 - loss: 0.9189
Epoch 17/50
688/688             1s 2ms/step -
accuracy: 0.6061 - loss: 0.9016
Epoch 18/50
688/688             1s 2ms/step -
accuracy: 0.6121 - loss: 0.8930
Epoch 19/50
688/688             1s 2ms/step -
accuracy: 0.6175 - loss: 0.8826
Epoch 20/50
688/688             1s 2ms/step -
accuracy: 0.6214 - loss: 0.8691
Epoch 21/50
688/688             1s 2ms/step -
accuracy: 0.6244 - loss: 0.8557
Epoch 22/50
688/688             1s 2ms/step -
accuracy: 0.6305 - loss: 0.8447
Epoch 23/50
688/688             1s 2ms/step -
accuracy: 0.6290 - loss: 0.8431
Epoch 24/50
688/688             1s 2ms/step -
```

```
accuracy: 0.6332 - loss: 0.8327
Epoch 25/50
688/688             1s 2ms/step -
accuracy: 0.6368 - loss: 0.8180
Epoch 26/50
688/688             1s 2ms/step -
accuracy: 0.6396 - loss: 0.8102
Epoch 27/50
688/688             1s 2ms/step -
accuracy: 0.6466 - loss: 0.8035
Epoch 28/50
688/688             1s 2ms/step -
accuracy: 0.6488 - loss: 0.7909
Epoch 29/50
688/688             1s 2ms/step -
accuracy: 0.6545 - loss: 0.7809
Epoch 30/50
688/688             1s 2ms/step -
accuracy: 0.6560 - loss: 0.7735
Epoch 31/50
688/688             1s 2ms/step -
accuracy: 0.6612 - loss: 0.7591
Epoch 32/50
688/688             1s 2ms/step -
accuracy: 0.6621 - loss: 0.7532
Epoch 33/50
688/688             1s 2ms/step -
accuracy: 0.6654 - loss: 0.7472
Epoch 34/50
688/688             1s 2ms/step -
accuracy: 0.6679 - loss: 0.7350
Epoch 35/50
688/688             1s 2ms/step -
accuracy: 0.6787 - loss: 0.7187
Epoch 36/50
688/688             1s 2ms/step -
accuracy: 0.6749 - loss: 0.7161
Epoch 37/50
688/688             1s 2ms/step -
accuracy: 0.6835 - loss: 0.7055
Epoch 38/50
688/688             1s 2ms/step -
accuracy: 0.6880 - loss: 0.6943
Epoch 39/50
688/688             1s 2ms/step -
accuracy: 0.6922 - loss: 0.6825
Epoch 40/50
688/688             1s 2ms/step -
```

```
accuracy: 0.6941 - loss: 0.6783
Epoch 41/50
688/688             1s 2ms/step -
accuracy: 0.6988 - loss: 0.6622
Epoch 42/50
688/688             1s 2ms/step -
accuracy: 0.6986 - loss: 0.6618
Epoch 43/50
688/688             1s 2ms/step -
accuracy: 0.7045 - loss: 0.6493
Epoch 44/50
688/688             1s 2ms/step -
accuracy: 0.7130 - loss: 0.6365
Epoch 45/50
688/688             1s 2ms/step -
accuracy: 0.7152 - loss: 0.6230
Epoch 46/50
688/688             1s 2ms/step -
accuracy: 0.7178 - loss: 0.6207
Epoch 47/50
688/688             1s 2ms/step -
accuracy: 0.7225 - loss: 0.6067
Epoch 48/50
688/688             1s 2ms/step -
accuracy: 0.7216 - loss: 0.6072
Epoch 49/50
688/688             1s 2ms/step -
accuracy: 0.7225 - loss: 0.6011
Epoch 50/50
688/688             1s 2ms/step -
accuracy: 0.7297 - loss: 0.5866
63/63             0s 1ms/step
Model: avg+median filtering results
Training Time: 130.540
acc: 0.6525
prec:0.7419
recall:0.6525
```

Performed Worse than on just the raw training data.

[12]:
```
avg_nn = Pipeline([
    ("avg14", FunctionTransformer(avg_2x2_pool)),
    ("flat", FunctionTransformer(flatten_data)),
    ("scale", MinMaxScaler()),
    ("model", clf)
])
```

```
results.append(train_eval_model(
    name="avg filtering results",
    model=avg_nn,
    x_tr=x_tr,
    x_va=x_va,
    y_tr=y_tr,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[tensorboard_cb, early_stopping_cb]
))
```

```
Epoch 1/50
```

```
c:\Users\samue\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
688/688            2s 2ms/step -
accuracy: 0.4170 - loss: 1.3052
Epoch 2/50
688/688            1s 2ms/step -
accuracy: 0.4988 - loss: 1.1916
Epoch 3/50
688/688            1s 2ms/step -
accuracy: 0.5324 - loss: 1.1346
Epoch 4/50
688/688            1s 2ms/step -
accuracy: 0.5550 - loss: 1.0843
Epoch 5/50
688/688            1s 2ms/step -
accuracy: 0.5625 - loss: 1.0466
Epoch 6/50
688/688            1s 2ms/step -
accuracy: 0.5769 - loss: 1.0123
Epoch 7/50
688/688            1s 2ms/step -
accuracy: 0.5883 - loss: 0.9860
Epoch 8/50
688/688            1s 2ms/step -
accuracy: 0.5928 - loss: 0.9618
Epoch 9/50
688/688            1s 2ms/step -
accuracy: 0.6036 - loss: 0.9371
Epoch 10/50
688/688            1s 2ms/step -
accuracy: 0.6095 - loss: 0.9224
```

```
Epoch 11/50
688/688             1s 2ms/step -
accuracy: 0.6091 - loss: 0.9069
Epoch 12/50
688/688             1s 2ms/step -
accuracy: 0.6173 - loss: 0.8968
Epoch 13/50
688/688             1s 2ms/step -
accuracy: 0.6229 - loss: 0.8775
Epoch 14/50
688/688             1s 2ms/step -
accuracy: 0.6290 - loss: 0.8639
Epoch 15/50
688/688             1s 2ms/step -
accuracy: 0.6339 - loss: 0.8507
Epoch 16/50
688/688             1s 2ms/step -
accuracy: 0.6350 - loss: 0.8401
Epoch 17/50
688/688             1s 2ms/step -
accuracy: 0.6497 - loss: 0.8200
Epoch 18/50
688/688             1s 2ms/step -
accuracy: 0.6470 - loss: 0.8080
Epoch 19/50
688/688             1s 2ms/step -
accuracy: 0.6516 - loss: 0.8008
Epoch 20/50
688/688             1s 2ms/step -
accuracy: 0.6605 - loss: 0.7833
Epoch 21/50
688/688             1s 2ms/step -
accuracy: 0.6639 - loss: 0.7770
Epoch 22/50
688/688             1s 2ms/step -
accuracy: 0.6680 - loss: 0.7639
Epoch 23/50
688/688             1s 2ms/step -
accuracy: 0.6759 - loss: 0.7500
Epoch 24/50
688/688             1s 2ms/step -
accuracy: 0.6781 - loss: 0.7360
Epoch 25/50
688/688             1s 2ms/step -
accuracy: 0.6828 - loss: 0.7278
Epoch 26/50
688/688             1s 2ms/step -
accuracy: 0.6845 - loss: 0.7165
```

```
Epoch 27/50
688/688             1s 2ms/step -
accuracy: 0.6879 - loss: 0.7083
Epoch 28/50
688/688             1s 2ms/step -
accuracy: 0.6971 - loss: 0.6956
Epoch 29/50
688/688             1s 2ms/step -
accuracy: 0.7012 - loss: 0.6781
Epoch 30/50
688/688             1s 2ms/step -
accuracy: 0.7048 - loss: 0.6693
Epoch 31/50
688/688             1s 2ms/step -
accuracy: 0.7111 - loss: 0.6611
Epoch 32/50
688/688             1s 2ms/step -
accuracy: 0.7127 - loss: 0.6483
Epoch 33/50
688/688             1s 2ms/step -
accuracy: 0.7125 - loss: 0.6396
Epoch 34/50
688/688             1s 2ms/step -
accuracy: 0.7233 - loss: 0.6269
Epoch 35/50
688/688             1s 2ms/step -
accuracy: 0.7227 - loss: 0.6229
Epoch 36/50
688/688             1s 2ms/step -
accuracy: 0.7294 - loss: 0.6080
Epoch 37/50
688/688             1s 2ms/step -
accuracy: 0.7363 - loss: 0.5978
Epoch 38/50
688/688             1s 2ms/step -
accuracy: 0.7362 - loss: 0.5943
Epoch 39/50
688/688             1s 2ms/step -
accuracy: 0.7419 - loss: 0.5753
Epoch 40/50
688/688             1s 2ms/step -
accuracy: 0.7456 - loss: 0.5604
Epoch 41/50
688/688             1s 2ms/step -
accuracy: 0.7535 - loss: 0.5509
Epoch 42/50
688/688             1s 2ms/step -
accuracy: 0.7550 - loss: 0.5443
```

```
Epoch 43/50
688/688             1s 2ms/step -
accuracy: 0.7563 - loss: 0.5435
Epoch 44/50
688/688             1s 2ms/step -
accuracy: 0.7688 - loss: 0.5154
Epoch 45/50
688/688             1s 2ms/step -
accuracy: 0.7667 - loss: 0.5142
Epoch 46/50
688/688             1s 2ms/step -
accuracy: 0.7713 - loss: 0.5057
Epoch 47/50
688/688             1s 2ms/step -
accuracy: 0.7725 - loss: 0.4941
Epoch 48/50
688/688             1s 2ms/step -
accuracy: 0.7735 - loss: 0.4902
Epoch 49/50
688/688             1s 2ms/step -
accuracy: 0.7795 - loss: 0.4848
Epoch 50/50
688/688             1s 2ms/step -
accuracy: 0.7824 - loss: 0.4741
63/63             0s 1ms/step
Model: avg filtering results
Training Time: 65.793
acc: 0.7070
prec:0.7511
recall:0.7070
```

Once again overall a worse performance. Preprocessing appears to be a possible detriment for this.

All perfrom roughly similar with a slight edge to no preprocessing at this moment.

Now lets test results on a wide and deep network. The goal of a wide and deep network is to both hopefully learn simple patterns that can be learned just from the input data. As well as complex patterns that can be learned from the deep path too.

[13]:
```python
def create_wide_and_deep(meta):
    '''Creates the wide & deep network.
    deep network is designed to be same as pervious networks
    so we can guage if the width actually adds anything to the network.
    '''
    X_shape = meta["X_shape_"]
    print(X_shape)

    inputs= tf.keras.layers.Input(shape=(X_shape[1],))
```

```
    hidden_layer_1 = tf.keras.layers.Dense(300, activation='relu')(inputs)
    hidden_layer_2 = tf.keras.layers.Dense(300,␣
 ↪activation='relu')(hidden_layer_1)

    concat_layer = tf.keras.layers.Concatenate()([inputs, hidden_layer_2])
    output_layer = tf.keras.layers.Dense(4, activation="softmax")(concat_layer)


    model = tf.keras.Model(inputs=inputs, outputs=output_layer)

    model.compile(optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy'])
    return model

clf_wide = KerasClassifier(model=create_wide_and_deep, epochs=50,␣
 ↪batch_size=32, verbose=1)

pipe_baseline_wide = Pipeline([
    ("Flatten", FunctionTransformer(flatten_data)),
    ("Scale", MinMaxScaler()),
    ("model", clf_wide)
])
```

[14]:
```
results.append(train_eval_model(
    name="Wide & Deep baseline model",
    model=pipe_baseline_wide,
    x_tr=x_tr,
    y_tr=y_tr,
    x_va=x_va,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb, tensorboard_cb]
))
```

```
(22000, 784)
Epoch 1/50
688/688             3s 3ms/step -
accuracy: 0.3962 - loss: 1.3213
Epoch 2/50
688/688             2s 3ms/step -
accuracy: 0.4812 - loss: 1.2080
Epoch 3/50
688/688             2s 3ms/step -
accuracy: 0.5206 - loss: 1.1427
Epoch 4/50
688/688             2s 3ms/step -
```

```
accuracy: 0.5466 - loss: 1.0883
Epoch 5/50
688/688             2s 3ms/step -
accuracy: 0.5606 - loss: 1.0495
Epoch 6/50
688/688             2s 3ms/step -
accuracy: 0.5735 - loss: 1.0196
Epoch 7/50
688/688             2s 3ms/step -
accuracy: 0.5898 - loss: 0.9853
Epoch 8/50
688/688             2s 3ms/step -
accuracy: 0.6055 - loss: 0.9587
Epoch 9/50
688/688             2s 3ms/step -
accuracy: 0.6135 - loss: 0.9366
Epoch 10/50
688/688             2s 3ms/step -
accuracy: 0.6205 - loss: 0.9106
Epoch 11/50
688/688             2s 3ms/step -
accuracy: 0.6260 - loss: 0.8946
Epoch 12/50
688/688             2s 3ms/step -
accuracy: 0.6332 - loss: 0.8793
Epoch 13/50
688/688             2s 3ms/step -
accuracy: 0.6427 - loss: 0.8614
Epoch 14/50
688/688             2s 3ms/step -
accuracy: 0.6483 - loss: 0.8463
Epoch 15/50
688/688             2s 3ms/step -
accuracy: 0.6515 - loss: 0.8343
Epoch 16/50
688/688             2s 3ms/step -
accuracy: 0.6591 - loss: 0.8167
Epoch 17/50
688/688             2s 3ms/step -
accuracy: 0.6580 - loss: 0.8156
Epoch 18/50
688/688             2s 3ms/step -
accuracy: 0.6691 - loss: 0.7942
Epoch 19/50
688/688             2s 3ms/step -
accuracy: 0.6730 - loss: 0.7863
Epoch 20/50
688/688             2s 3ms/step -
```

```
accuracy: 0.6708 - loss: 0.7747
Epoch 21/50
688/688             2s 3ms/step -
accuracy: 0.6801 - loss: 0.7677
Epoch 22/50
688/688             2s 3ms/step -
accuracy: 0.6825 - loss: 0.7520
Epoch 23/50
688/688             2s 3ms/step -
accuracy: 0.6858 - loss: 0.7443
Epoch 24/50
688/688             2s 3ms/step -
accuracy: 0.6895 - loss: 0.7339
Epoch 25/50
688/688             2s 3ms/step -
accuracy: 0.6914 - loss: 0.7323
Epoch 26/50
688/688             2s 3ms/step -
accuracy: 0.6902 - loss: 0.7199
Epoch 27/50
688/688             2s 3ms/step -
accuracy: 0.6987 - loss: 0.7129
Epoch 28/50
688/688             2s 3ms/step -
accuracy: 0.7039 - loss: 0.6955
Epoch 29/50
688/688             2s 3ms/step -
accuracy: 0.7054 - loss: 0.6900
Epoch 30/50
688/688             2s 3ms/step -
accuracy: 0.7100 - loss: 0.6810
Epoch 31/50
688/688             2s 3ms/step -
accuracy: 0.7147 - loss: 0.6725
Epoch 32/50
688/688             2s 3ms/step -
accuracy: 0.7161 - loss: 0.6658
Epoch 33/50
688/688             2s 3ms/step -
accuracy: 0.7189 - loss: 0.6629
Epoch 34/50
688/688             2s 3ms/step -
accuracy: 0.7273 - loss: 0.6386
Epoch 35/50
688/688             2s 2ms/step -
accuracy: 0.7257 - loss: 0.6403
Epoch 36/50
688/688             2s 3ms/step -
```

```
accuracy: 0.7279 - loss: 0.6352
Epoch 37/50
688/688             2s 3ms/step -
accuracy: 0.7310 - loss: 0.6302
Epoch 38/50
688/688             2s 2ms/step -
accuracy: 0.7315 - loss: 0.6246
Epoch 39/50
688/688             2s 3ms/step -
accuracy: 0.7320 - loss: 0.6192
Epoch 40/50
688/688             2s 3ms/step -
accuracy: 0.7400 - loss: 0.6046
Epoch 41/50
688/688             2s 3ms/step -
accuracy: 0.7435 - loss: 0.5978
Epoch 42/50
688/688             2s 3ms/step -
accuracy: 0.7420 - loss: 0.5959
Epoch 43/50
688/688             2s 3ms/step -
accuracy: 0.7425 - loss: 0.5877
Epoch 44/50
688/688             2s 3ms/step -
accuracy: 0.7507 - loss: 0.5751
Epoch 45/50
688/688             2s 3ms/step -
accuracy: 0.7523 - loss: 0.5645
Epoch 46/50
688/688             2s 3ms/step -
accuracy: 0.7578 - loss: 0.5605
Epoch 47/50
688/688             2s 2ms/step -
accuracy: 0.7587 - loss: 0.5566
Epoch 48/50
688/688             2s 2ms/step -
accuracy: 0.7626 - loss: 0.5411
Epoch 49/50
688/688             2s 3ms/step -
accuracy: 0.7621 - loss: 0.5397
Epoch 50/50
688/688             2s 2ms/step -
accuracy: 0.7589 - loss: 0.5565
63/63             0s 1ms/step
Model: Wide & Deep baseline model
Training Time: 94.969
acc: 0.6490
prec:0.7630
```

```
recall:0.6490
```

Very consistent results but nothing incredible.

[15]:
```
pipe_avg_wide = Pipeline([
    ("avg", FunctionTransformer(avg_2x2_pool)),
    ("Flatten", FunctionTransformer(flatten_data)),
    ("Scale", MinMaxScaler()),
    ("model", clf_wide)
])

results.append(train_eval_model(
    name="Wide & Deep Average",
    model=pipe_avg_wide,
    x_tr=x_tr,
    y_tr=y_tr,
    x_va=x_va,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb, tensorboard_cb]
))
```

```
(22000, 196)
Epoch 1/50
688/688             2s 2ms/step -
accuracy: 0.4198 - loss: 1.2971
Epoch 2/50
688/688             1s 2ms/step -
accuracy: 0.5029 - loss: 1.1801
Epoch 3/50
688/688             1s 2ms/step -
accuracy: 0.5350 - loss: 1.1186
Epoch 4/50
688/688             1s 2ms/step -
accuracy: 0.5505 - loss: 1.0821
Epoch 5/50
688/688             1s 2ms/step -
accuracy: 0.5670 - loss: 1.0475
Epoch 6/50
688/688             1s 2ms/step -
accuracy: 0.5729 - loss: 1.0176
Epoch 7/50
688/688             1s 2ms/step -
accuracy: 0.5837 - loss: 0.9903
Epoch 8/50
688/688             1s 2ms/step -
accuracy: 0.6015 - loss: 0.9607
Epoch 9/50
```

```
688/688            1s 2ms/step -
accuracy: 0.6052 - loss: 0.9473
Epoch 10/50
688/688            1s 2ms/step -
accuracy: 0.6101 - loss: 0.9213
Epoch 11/50
688/688            1s 2ms/step -
accuracy: 0.6169 - loss: 0.9063
Epoch 12/50
688/688            1s 2ms/step -
accuracy: 0.6288 - loss: 0.8910
Epoch 13/50
688/688            1s 2ms/step -
accuracy: 0.6321 - loss: 0.8726
Epoch 14/50
688/688            1s 2ms/step -
accuracy: 0.6407 - loss: 0.8574
Epoch 15/50
688/688            1s 2ms/step -
accuracy: 0.6408 - loss: 0.8475
Epoch 16/50
688/688            1s 2ms/step -
accuracy: 0.6462 - loss: 0.8300
Epoch 17/50
688/688            1s 2ms/step -
accuracy: 0.6500 - loss: 0.8163
Epoch 18/50
688/688            1s 2ms/step -
accuracy: 0.6546 - loss: 0.8111
Epoch 19/50
688/688            1s 2ms/step -
accuracy: 0.6572 - loss: 0.7889
Epoch 20/50
688/688            1s 2ms/step -
accuracy: 0.6703 - loss: 0.7755
Epoch 21/50
688/688            1s 2ms/step -
accuracy: 0.6720 - loss: 0.7679
Epoch 22/50
688/688            1s 2ms/step -
accuracy: 0.6696 - loss: 0.7582
Epoch 23/50
688/688            1s 2ms/step -
accuracy: 0.6759 - loss: 0.7411
Epoch 24/50
688/688            1s 2ms/step -
accuracy: 0.6844 - loss: 0.7301
Epoch 25/50
```

```
688/688             1s 2ms/step -
accuracy: 0.6878 - loss: 0.7184
Epoch 26/50
688/688             1s 2ms/step -
accuracy: 0.6971 - loss: 0.6969
Epoch 27/50
688/688             1s 2ms/step -
accuracy: 0.6955 - loss: 0.7001
Epoch 28/50
688/688             1s 2ms/step -
accuracy: 0.6995 - loss: 0.6835
Epoch 29/50
688/688             1s 2ms/step -
accuracy: 0.7090 - loss: 0.6727
Epoch 30/50
688/688             1s 2ms/step -
accuracy: 0.7078 - loss: 0.6647
Epoch 31/50
688/688             1s 2ms/step -
accuracy: 0.7147 - loss: 0.6426
Epoch 32/50
688/688             1s 2ms/step -
accuracy: 0.7178 - loss: 0.6327
Epoch 33/50
688/688             1s 2ms/step -
accuracy: 0.7209 - loss: 0.6263
Epoch 34/50
688/688             1s 2ms/step -
accuracy: 0.7275 - loss: 0.6183
Epoch 35/50
688/688             1s 2ms/step -
accuracy: 0.7360 - loss: 0.6037
Epoch 36/50
688/688             1s 2ms/step -
accuracy: 0.7350 - loss: 0.5902
Epoch 37/50
688/688             1s 2ms/step -
accuracy: 0.7373 - loss: 0.5851
Epoch 38/50
688/688             1s 2ms/step -
accuracy: 0.7445 - loss: 0.5705
Epoch 39/50
688/688             1s 2ms/step -
accuracy: 0.7515 - loss: 0.5552
Epoch 40/50
688/688             1s 2ms/step -
accuracy: 0.7540 - loss: 0.5487
Epoch 41/50
```

```
688/688             1s 2ms/step -
accuracy: 0.7574 - loss: 0.5371
Epoch 42/50
688/688             1s 2ms/step -
accuracy: 0.7617 - loss: 0.5321
Epoch 43/50
688/688             1s 2ms/step -
accuracy: 0.7669 - loss: 0.5189
Epoch 44/50
688/688             1s 2ms/step -
accuracy: 0.7706 - loss: 0.5069
Epoch 45/50
688/688             1s 2ms/step -
accuracy: 0.7728 - loss: 0.5003
Epoch 46/50
688/688             1s 2ms/step -
accuracy: 0.7790 - loss: 0.4859
Epoch 47/50
688/688             1s 2ms/step -
accuracy: 0.7849 - loss: 0.4717
Epoch 48/50
688/688             1s 2ms/step -
accuracy: 0.7889 - loss: 0.4699
Epoch 49/50
688/688             1s 2ms/step -
accuracy: 0.7900 - loss: 0.4612
Epoch 50/50
688/688             1s 2ms/step -
accuracy: 0.7938 - loss: 0.4542
63/63             0s 1ms/step
Model: Wide & Deep Average
Training Time: 64.602
acc: 0.6630
prec:0.7542
recall:0.6630
```

Best Accuracy So Far, but lower precision

[16]:
```
pipe_med_avg_wide = Pipeline([
    ("med", FunctionTransformer(median)),
    ("avg", FunctionTransformer(avg_2x2_pool)),
    ("Flatten", FunctionTransformer(flatten_data)),
    ("Scale", MinMaxScaler()),
    ("model", clf_wide)
])

results.append(train_eval_model(
```

```
    name="Wide & Deep Median + Avg model",
    model=pipe_med_avg_wide,
    x_tr=x_tr,
    y_tr=y_tr,
    x_va=x_va,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb, tensorboard_cb]
))
```

```
(22000, 196)
Epoch 1/50
688/688             2s 2ms/step -
accuracy: 0.4304 - loss: 1.3104
Epoch 2/50
688/688             1s 2ms/step -
accuracy: 0.4974 - loss: 1.2013
Epoch 3/50
688/688             1s 2ms/step -
accuracy: 0.5203 - loss: 1.1539
Epoch 4/50
688/688             1s 2ms/step -
accuracy: 0.5334 - loss: 1.1181
Epoch 5/50
688/688             1s 2ms/step -
accuracy: 0.5469 - loss: 1.0847
Epoch 6/50
688/688             1s 2ms/step -
accuracy: 0.5550 - loss: 1.0610
Epoch 7/50
688/688             1s 2ms/step -
accuracy: 0.5682 - loss: 1.0328
Epoch 8/50
688/688             1s 2ms/step -
accuracy: 0.5682 - loss: 1.0235
Epoch 9/50
688/688             1s 2ms/step -
accuracy: 0.5766 - loss: 1.0085
Epoch 10/50
688/688             1s 2ms/step -
accuracy: 0.5867 - loss: 0.9881
Epoch 11/50
688/688             1s 2ms/step -
accuracy: 0.5939 - loss: 0.9649
Epoch 12/50
688/688             1s 2ms/step -
accuracy: 0.5938 - loss: 0.9524
Epoch 13/50
```

```
688/688             1s 2ms/step -
accuracy: 0.5963 - loss: 0.9410
Epoch 14/50
688/688             1s 2ms/step -
accuracy: 0.6028 - loss: 0.9305
Epoch 15/50
688/688             1s 2ms/step -
accuracy: 0.6091 - loss: 0.9126
Epoch 16/50
688/688             1s 2ms/step -
accuracy: 0.6120 - loss: 0.9015
Epoch 17/50
688/688             1s 2ms/step -
accuracy: 0.6136 - loss: 0.8887
Epoch 18/50
688/688             1s 2ms/step -
accuracy: 0.6157 - loss: 0.8815
Epoch 19/50
688/688             1s 2ms/step -
accuracy: 0.6215 - loss: 0.8651
Epoch 20/50
688/688             1s 2ms/step -
accuracy: 0.6223 - loss: 0.8603
Epoch 21/50
688/688             1s 2ms/step -
accuracy: 0.6324 - loss: 0.8431
Epoch 22/50
688/688             1s 2ms/step -
accuracy: 0.6362 - loss: 0.8298
Epoch 23/50
688/688             1s 2ms/step -
accuracy: 0.6351 - loss: 0.8270
Epoch 24/50
688/688             1s 2ms/step -
accuracy: 0.6378 - loss: 0.8222
Epoch 25/50
688/688             1s 2ms/step -
accuracy: 0.6467 - loss: 0.8062
Epoch 26/50
688/688             1s 2ms/step -
accuracy: 0.6462 - loss: 0.7937
Epoch 27/50
688/688             1s 2ms/step -
accuracy: 0.6510 - loss: 0.7882
Epoch 28/50
688/688             1s 2ms/step -
accuracy: 0.6575 - loss: 0.7762
Epoch 29/50
```

```
688/688            1s 2ms/step -
accuracy: 0.6567 - loss: 0.7662
Epoch 30/50
688/688            1s 2ms/step -
accuracy: 0.6633 - loss: 0.7554
Epoch 31/50
688/688            1s 2ms/step -
accuracy: 0.6668 - loss: 0.7431
Epoch 32/50
688/688            1s 2ms/step -
accuracy: 0.6715 - loss: 0.7353
Epoch 33/50
688/688            1s 2ms/step -
accuracy: 0.6757 - loss: 0.7266
Epoch 34/50
688/688            1s 2ms/step -
accuracy: 0.6815 - loss: 0.7173
Epoch 35/50
688/688            1s 2ms/step -
accuracy: 0.6791 - loss: 0.7129
Epoch 36/50
688/688            1s 2ms/step -
accuracy: 0.6835 - loss: 0.6998
Epoch 37/50
688/688            1s 2ms/step -
accuracy: 0.6885 - loss: 0.6854
Epoch 38/50
688/688            1s 2ms/step -
accuracy: 0.6924 - loss: 0.6771
Epoch 39/50
688/688            1s 2ms/step -
accuracy: 0.6971 - loss: 0.6679
Epoch 40/50
688/688            1s 2ms/step -
accuracy: 0.7020 - loss: 0.6615
Epoch 41/50
688/688            1s 2ms/step -
accuracy: 0.7052 - loss: 0.6432
Epoch 42/50
688/688            1s 2ms/step -
accuracy: 0.7068 - loss: 0.6464
Epoch 43/50
688/688            1s 2ms/step -
accuracy: 0.7105 - loss: 0.6343
Epoch 44/50
688/688            1s 2ms/step -
accuracy: 0.7143 - loss: 0.6287
Epoch 45/50
```

```
688/688             1s 2ms/step -
accuracy: 0.7201 - loss: 0.6140
Epoch 46/50
688/688             1s 2ms/step -
accuracy: 0.7244 - loss: 0.6018
Epoch 47/50
688/688             1s 2ms/step -
accuracy: 0.7259 - loss: 0.6052
Epoch 48/50
688/688             1s 2ms/step -
accuracy: 0.7262 - loss: 0.5957
Epoch 49/50
688/688             1s 2ms/step -
accuracy: 0.7325 - loss: 0.5798
Epoch 50/50
688/688             1s 2ms/step -
accuracy: 0.7383 - loss: 0.5696
63/63             0s 1ms/step
Model: Wide & Deep Median + Avg model
Training Time: 127.973
acc: 0.6650
prec:0.7434
recall:0.6650
```

[17]:
```python
def create_wide_and_deep_subsample(meta, wide_frac=0.5, deep_frac=0.5, seed=42):
    X_shape_ = meta["X_shape_"]
    n_features = X_shape_[1]

    rng = np.random.RandomState(seed)
    wide_idx = rng.choice(n_features, size=int(n_features * wide_frac),␣
 ↪replace=False)
    deep_idx = rng.choice(n_features, size=int(n_features * deep_frac),␣
 ↪replace=False)

    inputs = tf.keras.layers.Input(shape=(n_features,))

    wide_input = tf.keras.layers.Lambda(lambda x: tf.gather(x, wide_idx,␣
 ↪axis=1))(inputs)
    deep_input = tf.keras.layers.Lambda(lambda x: tf.gather(x, deep_idx,␣
 ↪axis=1))(inputs)

    # Deep path
    deep = tf.keras.layers.Dense(300, activation="relu")(deep_input)
    deep = tf.keras.layers.Dense(300, activation="relu")(deep)

    # Concatenate
```

```
    concat = tf.keras.layers.Concatenate()([wide_input, deep])

    output = tf.keras.layers.Dense(4, activation="softmax")(concat)

    model = tf.keras.Model(inputs=inputs, outputs=output)
    model.compile(
        optimizer="adam",
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
        metrics=["accuracy"]
    )
    return model

clf_wide_subsample = KerasClassifier(model=create_wide_and_deep_subsample,␣
 ↪epochs=50, batch_size=32, verbose=1)

pipe_wide_subsample_base = Pipeline([
    ("flat", flattener),
    ("scale", MinMaxScaler()),
    ("model", clf_wide_subsample)
])

results.append(train_eval_model(
    name="Wide & Deep with feature subsampling",
    model=pipe_wide_subsample_base,
    x_tr=x_tr,
    x_va=x_va,
    y_tr=y_tr,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb, tensorboard_cb]
))
```

```
WARNING:tensorflow:From c:\Users\samue\anaconda3\Lib\site-
packages\keras\src\backend\tensorflow\core.py:232: The name tf.placeholder is
deprecated. Please use tf.compat.v1.placeholder instead.

Epoch 1/50
688/688             2s 2ms/step -
accuracy: 0.3925 - loss: 1.3209
Epoch 2/50
688/688             2s 2ms/step -
accuracy: 0.4680 - loss: 1.2159
Epoch 3/50
688/688             2s 2ms/step -
accuracy: 0.5079 - loss: 1.1575
Epoch 4/50
688/688             2s 2ms/step -
accuracy: 0.5418 - loss: 1.0993
```

```
Epoch 5/50
688/688             2s 2ms/step -
accuracy: 0.5638 - loss: 1.0578
Epoch 6/50
688/688             2s 2ms/step -
accuracy: 0.5758 - loss: 1.0148
Epoch 7/50
688/688             2s 2ms/step -
accuracy: 0.6050 - loss: 0.9834
Epoch 8/50
688/688             2s 2ms/step -
accuracy: 0.6099 - loss: 0.9478
Epoch 9/50
688/688             2s 2ms/step -
accuracy: 0.6230 - loss: 0.9266
Epoch 10/50
688/688             2s 2ms/step -
accuracy: 0.6300 - loss: 0.9010
Epoch 11/50
688/688             2s 2ms/step -
accuracy: 0.6398 - loss: 0.8778
Epoch 12/50
688/688             2s 2ms/step -
accuracy: 0.6430 - loss: 0.8687
Epoch 13/50
688/688             2s 2ms/step -
accuracy: 0.6523 - loss: 0.8437
Epoch 14/50
688/688             2s 2ms/step -
accuracy: 0.6588 - loss: 0.8236
Epoch 15/50
688/688             2s 2ms/step -
accuracy: 0.6637 - loss: 0.8099
Epoch 16/50
688/688             2s 2ms/step -
accuracy: 0.6660 - loss: 0.7954
Epoch 17/50
688/688             2s 2ms/step -
accuracy: 0.6721 - loss: 0.7782
Epoch 18/50
688/688             2s 2ms/step -
accuracy: 0.6782 - loss: 0.7632
Epoch 19/50
688/688             1s 2ms/step -
accuracy: 0.6862 - loss: 0.7488
Epoch 20/50
688/688             1s 2ms/step -
accuracy: 0.6890 - loss: 0.7363
```

```
Epoch 21/50
688/688             2s 2ms/step -
accuracy: 0.6912 - loss: 0.7313
Epoch 22/50
688/688             2s 2ms/step -
accuracy: 0.7013 - loss: 0.7066
Epoch 23/50
688/688             1s 2ms/step -
accuracy: 0.7058 - loss: 0.6935
Epoch 24/50
688/688             2s 2ms/step -
accuracy: 0.7068 - loss: 0.6835
Epoch 25/50
688/688             2s 2ms/step -
accuracy: 0.7120 - loss: 0.6699
Epoch 26/50
688/688             2s 2ms/step -
accuracy: 0.7192 - loss: 0.6581
Epoch 27/50
688/688             2s 2ms/step -
accuracy: 0.7215 - loss: 0.6475
Epoch 28/50
688/688             2s 2ms/step -
accuracy: 0.7267 - loss: 0.6374
Epoch 29/50
688/688             1s 2ms/step -
accuracy: 0.7327 - loss: 0.6208
Epoch 30/50
688/688             1s 2ms/step -
accuracy: 0.7319 - loss: 0.6175
Epoch 31/50
688/688             1s 2ms/step -
accuracy: 0.7421 - loss: 0.5948
Epoch 32/50
688/688             2s 2ms/step -
accuracy: 0.7450 - loss: 0.5898
Epoch 33/50
688/688             1s 2ms/step -
accuracy: 0.7453 - loss: 0.5796
Epoch 34/50
688/688             2s 2ms/step -
accuracy: 0.7536 - loss: 0.5669
Epoch 35/50
688/688             1s 2ms/step -
accuracy: 0.7546 - loss: 0.5595
Epoch 36/50
688/688             2s 2ms/step -
accuracy: 0.7640 - loss: 0.5440
```

```
Epoch 37/50
688/688             2s 2ms/step -
accuracy: 0.7621 - loss: 0.5423
Epoch 38/50
688/688             2s 2ms/step -
accuracy: 0.7634 - loss: 0.5339
Epoch 39/50
688/688             2s 2ms/step -
accuracy: 0.7689 - loss: 0.5212
Epoch 40/50
688/688             2s 2ms/step -
accuracy: 0.7756 - loss: 0.5108
Epoch 41/50
688/688             2s 2ms/step -
accuracy: 0.7789 - loss: 0.5020
Epoch 42/50
688/688             1s 2ms/step -
accuracy: 0.7813 - loss: 0.4970
Epoch 43/50
688/688             2s 2ms/step -
accuracy: 0.7908 - loss: 0.4803
Epoch 44/50
688/688             2s 2ms/step -
accuracy: 0.7893 - loss: 0.4705
Epoch 45/50
688/688             2s 2ms/step -
accuracy: 0.7977 - loss: 0.4645
Epoch 46/50
688/688             2s 2ms/step -
accuracy: 0.7982 - loss: 0.4548
Epoch 47/50
688/688             1s 2ms/step -
accuracy: 0.7999 - loss: 0.4524
Epoch 48/50
688/688             1s 2ms/step -
accuracy: 0.8052 - loss: 0.4350
Epoch 49/50
688/688             2s 2ms/step -
accuracy: 0.8095 - loss: 0.4307
Epoch 50/50
688/688             2s 2ms/step -
accuracy: 0.8116 - loss: 0.4195
63/63             0s 2ms/step
Model: Wide & Deep with feature subsampling
Training Time: 78.993
acc: 0.6775
prec:0.7505
recall:0.6775
```

[18]:
```
def prep_for_cnn(X):
    X = X.astype("float32") / 255.0
    X = X.reshape(X.shape[0], X.shape[1], X.shape[2], 1)
    return X
```

[19]:
```
#we first need to make sure the data is ready for a cnn (see utils helper)
x_tr_cnn = prep_for_cnn(x_tr) #train
x_va_cnn = prep_for_cnn(x_va) #validation
```

[20]:
```
from tensorflow import keras

def make_cnn():
    model = keras.Sequential([
        keras.layers.Input(shape=(28, 28, 1)),
        keras.layers.Conv2D(64, kernel_size=3, activation="relu"),
        keras.layers.MaxPooling2D(pool_size=2), #2x2
        keras.layers.Conv2D(32, kernel_size=3, activation="relu"),
        keras.layers.MaxPooling2D(pool_size=2),
        keras.layers.Flatten(),
        keras.layers.Dense(32, activation="relu"),
        keras.layers.Dense(4, activation="softmax")
    ])

    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

[64]:
```
import time
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score

def train_eval_cnn(name, model, x_tr, y_tr, x_va, y_va, sample_weights,␣
 ↪callbacks=None):
    t1 = time.time()

    history = model.fit(
        x_tr, y_tr,
        validation_data=(x_va, y_va),
        epochs=50,
        batch_size=32,
        sample_weight=sample_weights,
        callbacks=callbacks,
```

```
        verbose=1
    )

    t2 = time.time()

    probs = model.predict(x_va)
    pred = probs.argmax(axis=1)

    acc = accuracy_score(y_va, pred)
    prec = precision_score(y_va, pred, average="weighted")
    rec = recall_score(y_va, pred, average="weighted")
    f1 = f1_score(y_va, pred, average="weighted")
    train_time = t2-t1

    print(f"Model: {name}")
    print(f"Training Time: {t2-t1}")
    print(f"acc: {acc}")
    print(f"prec: {prec}")
    print(f"recall: {rec}")

    return history, {
        "name": name,
        "time": round(train_time, 3),
        "acc": round(acc, 4),
        "prec": round(prec, 4),
        "recall": round(rec, 4),
        "f1": f1
    }
```

[65]:
```
cnn_model = make_cnn()
cnn_history, cnn_result = train_eval_cnn(
    name="Simple CNN",
    model=cnn_model, #using the model we created
    x_tr=x_tr_cnn,
    y_tr=y_tr,
    x_va=x_va_cnn,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb]
)

results.append(cnn_result)
```

```
Epoch 1/50
688/688             5s 7ms/step -
accuracy: 0.5729 - loss: 1.1531 - val_accuracy: 0.6545 - val_loss: 0.8564
Epoch 2/50
688/688             4s 6ms/step -
```

```
accuracy: 0.6725 - loss: 0.8972 - val_accuracy: 0.6570 - val_loss: 0.8500
Epoch 3/50
688/688             5s 7ms/step -
accuracy: 0.6980 - loss: 0.8098 - val_accuracy: 0.7265 - val_loss: 0.7373
Epoch 4/50
688/688             5s 7ms/step -
accuracy: 0.7110 - loss: 0.7569 - val_accuracy: 0.6610 - val_loss: 0.8091
Epoch 5/50
688/688             5s 7ms/step -
accuracy: 0.7197 - loss: 0.7283 - val_accuracy: 0.7555 - val_loss: 0.6367
Epoch 6/50
688/688             5s 7ms/step -
accuracy: 0.7310 - loss: 0.6972 - val_accuracy: 0.7680 - val_loss: 0.6112
Epoch 7/50
688/688             5s 7ms/step -
accuracy: 0.7389 - loss: 0.6721 - val_accuracy: 0.5615 - val_loss: 0.9077
Epoch 8/50
688/688             5s 7ms/step -
accuracy: 0.7405 - loss: 0.6545 - val_accuracy: 0.7800 - val_loss: 0.5875
Epoch 9/50
688/688             5s 7ms/step -
accuracy: 0.7486 - loss: 0.6359 - val_accuracy: 0.7725 - val_loss: 0.5839
Epoch 10/50
688/688             5s 7ms/step -
accuracy: 0.7562 - loss: 0.6194 - val_accuracy: 0.7590 - val_loss: 0.5982
63/63             0s 3ms/step
Model: Simple CNN
Training Time: 47.55161905288696
acc: 0.6545
prec: 0.7605922556462199
recall: 0.6545
```

[66]:
```python
def prep_median_for_cnn(X):
    X_med = median(X)
    X_med = X_med.astype("float32") / 255.0
    X_med = X_med.reshape(X_med.shape[0], X_med.shape[1], X_med.shape[2], 1)
    return X_med
```

[67]:
```python
x_tr_cnn_med = prep_median_for_cnn(x_tr)
x_va_cnn_med = prep_median_for_cnn(x_va)
```

[83]:
```python
cnn_med_model = make_cnn()
cnn_med_history, cnn_med_result = train_eval_cnn(
    "Median CNN",
    cnn_med_model,
    x_tr_cnn_med, y_tr,
    x_va_cnn_med, y_va,
```

```
    sample_weights=sample_weights
)

results.append(cnn_med_result)
```

```
Epoch 1/50
688/688             5s 7ms/step -
accuracy: 0.5282 - loss: 1.2164 - val_accuracy: 0.6620 - val_loss: 0.9174
Epoch 2/50
688/688             4s 6ms/step -
accuracy: 0.6278 - loss: 0.9817 - val_accuracy: 0.6990 - val_loss: 0.8255
Epoch 3/50
688/688             4s 6ms/step -
accuracy: 0.6520 - loss: 0.8997 - val_accuracy: 0.6820 - val_loss: 0.8470
Epoch 4/50
688/688             4s 6ms/step -
accuracy: 0.6705 - loss: 0.8554 - val_accuracy: 0.6125 - val_loss: 0.8659
Epoch 5/50
688/688             4s 6ms/step -
accuracy: 0.6764 - loss: 0.8202 - val_accuracy: 0.6620 - val_loss: 0.7945
Epoch 6/50
688/688             5s 7ms/step -
accuracy: 0.6830 - loss: 0.7889 - val_accuracy: 0.6735 - val_loss: 0.7852
Epoch 7/50
688/688             5s 7ms/step -
accuracy: 0.6952 - loss: 0.7624 - val_accuracy: 0.5675 - val_loss: 0.9277
Epoch 8/50
688/688             5s 7ms/step -
accuracy: 0.7036 - loss: 0.7420 - val_accuracy: 0.5805 - val_loss: 0.8743
Epoch 9/50
688/688             5s 7ms/step -
accuracy: 0.7059 - loss: 0.7238 - val_accuracy: 0.7000 - val_loss: 0.7088
Epoch 10/50
688/688             5s 7ms/step -
accuracy: 0.7115 - loss: 0.7103 - val_accuracy: 0.6675 - val_loss: 0.7695
Epoch 11/50
688/688             5s 7ms/step -
accuracy: 0.7212 - loss: 0.6853 - val_accuracy: 0.7010 - val_loss: 0.7304
Epoch 12/50
688/688             5s 7ms/step -
accuracy: 0.7225 - loss: 0.6754 - val_accuracy: 0.7200 - val_loss: 0.6901
Epoch 13/50
688/688             5s 7ms/step -
accuracy: 0.7305 - loss: 0.6614 - val_accuracy: 0.6930 - val_loss: 0.7568
Epoch 14/50
688/688             5s 7ms/step -
accuracy: 0.7357 - loss: 0.6455 - val_accuracy: 0.7125 - val_loss: 0.7066
Epoch 15/50
```

```
688/688             5s 7ms/step -
accuracy: 0.7446 - loss: 0.6247 - val_accuracy: 0.7075 - val_loss: 0.7132
Epoch 16/50
688/688             5s 7ms/step -
accuracy: 0.7500 - loss: 0.6150 - val_accuracy: 0.6925 - val_loss: 0.7517
Epoch 17/50
688/688             5s 7ms/step -
accuracy: 0.7542 - loss: 0.6011 - val_accuracy: 0.6960 - val_loss: 0.7411
Epoch 18/50
688/688             5s 7ms/step -
accuracy: 0.7610 - loss: 0.5867 - val_accuracy: 0.6525 - val_loss: 0.8055
Epoch 19/50
688/688             5s 7ms/step -
accuracy: 0.7614 - loss: 0.5785 - val_accuracy: 0.6840 - val_loss: 0.7612
Epoch 20/50
688/688             5s 7ms/step -
accuracy: 0.7712 - loss: 0.5565 - val_accuracy: 0.7250 - val_loss: 0.7345
Epoch 21/50
688/688             5s 7ms/step -
accuracy: 0.7721 - loss: 0.5502 - val_accuracy: 0.7260 - val_loss: 0.7145
Epoch 22/50
688/688             5s 7ms/step -
accuracy: 0.7757 - loss: 0.5390 - val_accuracy: 0.7330 - val_loss: 0.6873
Epoch 23/50
688/688             5s 7ms/step -
accuracy: 0.7801 - loss: 0.5286 - val_accuracy: 0.7555 - val_loss: 0.6583
Epoch 24/50
688/688             5s 7ms/step -
accuracy: 0.7856 - loss: 0.5115 - val_accuracy: 0.7265 - val_loss: 0.7473
Epoch 25/50
688/688             5s 7ms/step -
accuracy: 0.7884 - loss: 0.5077 - val_accuracy: 0.7340 - val_loss: 0.6832
Epoch 26/50
688/688             5s 7ms/step -
accuracy: 0.7913 - loss: 0.4958 - val_accuracy: 0.7060 - val_loss: 0.7616
Epoch 27/50
688/688             5s 7ms/step -
accuracy: 0.7950 - loss: 0.4882 - val_accuracy: 0.7300 - val_loss: 0.7013
Epoch 28/50
688/688             5s 7ms/step -
accuracy: 0.7975 - loss: 0.4765 - val_accuracy: 0.7100 - val_loss: 0.7415
Epoch 29/50
688/688             5s 7ms/step -
accuracy: 0.8020 - loss: 0.4643 - val_accuracy: 0.6945 - val_loss: 0.7869
Epoch 30/50
688/688             5s 7ms/step -
accuracy: 0.8037 - loss: 0.4585 - val_accuracy: 0.7380 - val_loss: 0.7241
Epoch 31/50
```

```
688/688            5s 7ms/step -
accuracy: 0.8109 - loss: 0.4443 - val_accuracy: 0.7140 - val_loss: 0.7574
Epoch 32/50
688/688            5s 7ms/step -
accuracy: 0.8123 - loss: 0.4362 - val_accuracy: 0.7410 - val_loss: 0.7085
Epoch 33/50
688/688            5s 7ms/step -
accuracy: 0.8134 - loss: 0.4355 - val_accuracy: 0.7195 - val_loss: 0.7473
Epoch 34/50
688/688            5s 7ms/step -
accuracy: 0.8183 - loss: 0.4234 - val_accuracy: 0.7490 - val_loss: 0.7065
Epoch 35/50
688/688            5s 7ms/step -
accuracy: 0.8248 - loss: 0.4124 - val_accuracy: 0.7085 - val_loss: 0.7914
Epoch 36/50
688/688            5s 7ms/step -
accuracy: 0.8273 - loss: 0.4013 - val_accuracy: 0.7095 - val_loss: 0.7918
Epoch 37/50
688/688            5s 7ms/step -
accuracy: 0.8318 - loss: 0.3931 - val_accuracy: 0.7610 - val_loss: 0.6861
Epoch 38/50
688/688            5s 7ms/step -
accuracy: 0.8308 - loss: 0.3863 - val_accuracy: 0.7275 - val_loss: 0.7676
Epoch 39/50
688/688            5s 7ms/step -
accuracy: 0.8375 - loss: 0.3739 - val_accuracy: 0.7385 - val_loss: 0.7753
Epoch 40/50
688/688            5s 7ms/step -
accuracy: 0.8352 - loss: 0.3741 - val_accuracy: 0.7500 - val_loss: 0.7400
Epoch 41/50
688/688            5s 7ms/step -
accuracy: 0.8413 - loss: 0.3636 - val_accuracy: 0.7390 - val_loss: 0.7706
Epoch 42/50
688/688            5s 7ms/step -
accuracy: 0.8461 - loss: 0.3508 - val_accuracy: 0.7550 - val_loss: 0.7544
Epoch 43/50
688/688            5s 7ms/step -
accuracy: 0.8464 - loss: 0.3484 - val_accuracy: 0.7360 - val_loss: 0.7720
Epoch 44/50
688/688            5s 7ms/step -
accuracy: 0.8490 - loss: 0.3422 - val_accuracy: 0.7550 - val_loss: 0.7734
Epoch 45/50
688/688            5s 7ms/step -
accuracy: 0.8505 - loss: 0.3354 - val_accuracy: 0.7755 - val_loss: 0.7184
Epoch 46/50
688/688            5s 7ms/step -
accuracy: 0.8562 - loss: 0.3281 - val_accuracy: 0.7265 - val_loss: 0.8356
Epoch 47/50
```

```
688/688             5s 7ms/step -
accuracy: 0.8574 - loss: 0.3219 - val_accuracy: 0.7355 - val_loss: 0.8114
Epoch 48/50
688/688             5s 7ms/step -
accuracy: 0.8577 - loss: 0.3171 - val_accuracy: 0.7455 - val_loss: 0.7977
Epoch 49/50
688/688             5s 7ms/step -
accuracy: 0.8645 - loss: 0.3089 - val_accuracy: 0.7690 - val_loss: 0.7495
Epoch 50/50
688/688             5s 7ms/step -
accuracy: 0.8664 - loss: 0.3018 - val_accuracy: 0.7540 - val_loss: 0.8010
63/63             0s 3ms/step
Model: Median CNN
Training Time: 229.6866500377655
acc: 0.754
prec: 0.7988533851793009
recall: 0.754
```

The wide and deep model performed slightly better than the baseline model. Interestingly there seems to be a percision recall trade off between averaging for this case. When averaging to reduce dimensionality the precision dropped but the recall increased. I would interpret this as the model sort of being more 'certain' or 'aggressive' in it's guessing leading to more total positive cases captured on average per class but also less overall accuracy on those guesses. Ultimately we should take both these into consideration especially when acknowledging that our models are tuned for Accuracy (Marco Recall).

Let us now look towards tuning an existing model and observe the results there. First we will explore general ideas in model architecture. Number of layers, number of neurons, learning rate, and basic optimimizer choices. This can help us get more insight into what kind of model architectures are doing well for this problem.

[26]:
```
import keras_tuner as kt

def build_tuned_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,␣
 ↪sampling="log")

    optimizer = hp.Choice("optimizer", values=["sgd", "adam"])
    avg = hp.Choice("avg", values=[True, False])
    pool = hp.Int("pool_size", min_value=2, max_value=4)
    activation = hp.Choice("activation", values=["relu", "leaky_relu", "elu",␣
 ↪"selu"])
    batch_norm = hp.Choice("batch_norm", values=[True, False])
    decay_rate = hp.Float("decay_rate", min_value=0.9, max_value=0.99, step=0.
 ↪01)

    if optimizer == "sgd":
```

```
        optimizer = tf.keras.optimizers.SGD(
            learning_rate=tf.keras.optimizers.schedules.ExponentialDecay(
                initial_learning_rate=learning_rate,
                decay_steps=100,
                decay_rate=decay_rate,
            )
        )
    else:
        optimizer = tf.keras.optimizers.Adam(
            learning_rate=tf.keras.optimizers.schedules.ExponentialDecay(
                initial_learning_rate=learning_rate,
                decay_steps=100,
                decay_rate=decay_rate,
            )
        )

    model = tf.keras.Sequential()

    if avg:
        model.add(tf.keras.layers.Reshape((28, 28, 1))) #Added because 4d data␣
↪is expected for average pooling
        model.add(tf.keras.layers.AveragePooling2D(pool_size=(pool, pool)))

    model.add(tf.keras.layers.Rescaling(scale=1.0 / 255)) #Our MinMaxScaler in␣
↪tensorflow format

    model.add(tf.keras.layers.Flatten())

    dropout_rate = hp.Float("dropout", min_value=0.0, max_value=0.5, step=0.1)

    for i in range(n_hidden):
        n_neurons = hp.Int(f"n_neurons{i}", min_value=16, max_value=256)
        model.add(tf.keras.layers.Dense(n_neurons, activation=activation))
        if batch_norm:
            model.add(tf.keras.layers.BatchNormalization())
        if dropout_rate > 0:
            model.add(tf.keras.layers.Dropout(dropout_rate))
    model.add(tf.keras.layers.Dense(4, activation="softmax"))
    model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,␣
↪metrics=["accuracy"])
    return model
```

This is a pretty deep tuning search. so lets break it down. 1. This allows us first and formost give a general idea of what model architecture is preferred (deep path vs shallow, wide layers vs small layers) 2. Control our preprocessing step through the averaging choice. 3. Have a preliminary search over activation functions & optimizers 4. Introduce batch normalization & dropout rate if it is beneficial.

[ ]:

[82]:
```
tuner = kt.RandomSearch(
    build_tuned_model,
    objective="val_accuracy",
    max_trials=50,
    directory="tuning_results",
    project_name="my_rnd_search",
)
'''
tuner.search(
    x_tr, y_tr,
    epochs=50,
    validation_data=(x_va, y_va),
    callbacks=[
        early_stopping_cb,
        tensorboard_cb
    ]
)
'''

def train_eval_no_wrapper(name, model, x_tr, y_tr, x_va, y_va, sample_weights,␣
 ↪callbacks=[]):
    '''trains and evaluates the model'''
    t1 = time.time()
    model.fit(x_tr, y_tr, epochs=50, sample_weight=sample_weights,␣
 ↪callbacks=callbacks)
    t2 = time.time()

    probs = model.predict(x_va)
    pred = probs.argmax(axis=1)

    acc    = accuracy_score(y_va, pred)
    prec   = precision_score(y_va, pred, average="weighted", zero_division=1)
    recall = recall_score(y_va, pred, average="weighted", zero_division=1)
    f1 = f1_score(y_va, pred, average="weighted", zero_division=1)

    print(f"Model: {name}")
    print(f"Training Time: {t2-t1:.3f}")
    print(f'acc: {acc:.4f}')
    print(f'prec:{prec:.4f}')
    print(f'recall:{recall:.4f}\n')

    return {"name": name, "time":t2-t1, "acc": acc, "prec":prec, "recall":
 ↪recall, "f1": f1}

best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]
```

```
best_model =tuner.hypermodel.build(best_hp)

results.append(train_eval_no_wrapper(
    name="Best Random Search",
    model=best_model,
    x_tr=x_tr,
    y_tr=y_tr,
    x_va=x_va,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=early_stopping_cb
))
```

```
Reloading Tuner from tuning_results\my_rnd_search\tuner0.json
Epoch 1/50
688/688             8s 4ms/step -
accuracy: 0.3627 - loss: 1.3981
Epoch 2/50
 37/688             2s 4ms/step -
accuracy: 0.3783 - loss: 1.1962

c:\Users\samue\anaconda3\Lib\site-
packages\keras\src\callbacks\early_stopping.py:99: UserWarning: Early stopping
conditioned on metric `val_loss` which is not available. Available metrics are:
accuracy,loss
  current = self.get_monitor_value(logs)

688/688             3s 4ms/step -
accuracy: 0.4420 - loss: 1.2384
Epoch 3/50
688/688             3s 4ms/step -
accuracy: 0.4876 - loss: 1.1682
Epoch 4/50
688/688             3s 4ms/step -
accuracy: 0.5215 - loss: 1.1108
Epoch 5/50
688/688             3s 4ms/step -
accuracy: 0.5432 - loss: 1.0599
Epoch 6/50
688/688             3s 4ms/step -
accuracy: 0.5567 - loss: 1.0262
Epoch 7/50
688/688             3s 4ms/step -
accuracy: 0.5769 - loss: 0.9916
Epoch 8/50
688/688             3s 4ms/step -
accuracy: 0.5974 - loss: 0.9477
Epoch 9/50
688/688             3s 4ms/step -
```

```
accuracy: 0.5981 - loss: 0.9160
Epoch 10/50
688/688             3s 4ms/step -
accuracy: 0.6101 - loss: 0.8855
Epoch 11/50
688/688             3s 4ms/step -
accuracy: 0.6218 - loss: 0.8657
Epoch 12/50
688/688             3s 4ms/step -
accuracy: 0.6294 - loss: 0.8454
Epoch 13/50
688/688             3s 4ms/step -
accuracy: 0.6337 - loss: 0.8177
Epoch 14/50
688/688             3s 4ms/step -
accuracy: 0.6429 - loss: 0.8064
Epoch 15/50
688/688             3s 4ms/step -
accuracy: 0.6460 - loss: 0.7936
Epoch 16/50
688/688             3s 4ms/step -
accuracy: 0.6466 - loss: 0.7775
Epoch 17/50
688/688             3s 4ms/step -
accuracy: 0.6507 - loss: 0.7692
Epoch 18/50
688/688             3s 4ms/step -
accuracy: 0.6567 - loss: 0.7578
Epoch 19/50
688/688             3s 4ms/step -
accuracy: 0.6599 - loss: 0.7482
Epoch 20/50
688/688             3s 4ms/step -
accuracy: 0.6660 - loss: 0.7415
Epoch 21/50
688/688             3s 4ms/step -
accuracy: 0.6685 - loss: 0.7423
Epoch 22/50
688/688             3s 4ms/step -
accuracy: 0.6649 - loss: 0.7346
Epoch 23/50
688/688             3s 4ms/step -
accuracy: 0.6654 - loss: 0.7303
Epoch 24/50
688/688             3s 4ms/step -
accuracy: 0.6698 - loss: 0.7303
Epoch 25/50
688/688             3s 4ms/step -
```

```
accuracy: 0.6730 - loss: 0.7267
Epoch 26/50
688/688             3s 4ms/step -
accuracy: 0.6709 - loss: 0.7233
Epoch 27/50
688/688             3s 4ms/step -
accuracy: 0.6745 - loss: 0.7219
Epoch 28/50
688/688             3s 4ms/step -
accuracy: 0.6709 - loss: 0.7271
Epoch 29/50
688/688             3s 4ms/step -
accuracy: 0.6749 - loss: 0.7249
Epoch 30/50
688/688             3s 4ms/step -
accuracy: 0.6755 - loss: 0.7210
Epoch 31/50
688/688             3s 4ms/step -
accuracy: 0.6748 - loss: 0.7209
Epoch 32/50
688/688             3s 4ms/step -
accuracy: 0.6707 - loss: 0.7214
Epoch 33/50
688/688             3s 4ms/step -
accuracy: 0.6734 - loss: 0.7206
Epoch 34/50
688/688             3s 4ms/step -
accuracy: 0.6739 - loss: 0.7138
Epoch 35/50
688/688             3s 4ms/step -
accuracy: 0.6731 - loss: 0.7185
Epoch 36/50
688/688             3s 4ms/step -
accuracy: 0.6710 - loss: 0.7254
Epoch 37/50
688/688             3s 4ms/step -
accuracy: 0.6740 - loss: 0.7139
Epoch 38/50
688/688             3s 4ms/step -
accuracy: 0.6772 - loss: 0.7176
Epoch 39/50
688/688             3s 4ms/step -
accuracy: 0.6781 - loss: 0.7129
Epoch 40/50
688/688             3s 4ms/step -
accuracy: 0.6689 - loss: 0.7303
Epoch 41/50
688/688             3s 4ms/step -
```

```
accuracy: 0.6739 - loss: 0.7205
Epoch 42/50
688/688             3s 4ms/step -
accuracy: 0.6715 - loss: 0.7218
Epoch 43/50
688/688             3s 4ms/step -
accuracy: 0.6716 - loss: 0.7198
Epoch 44/50
688/688             3s 4ms/step -
accuracy: 0.6733 - loss: 0.7192
Epoch 45/50
688/688             3s 4ms/step -
accuracy: 0.6754 - loss: 0.7167
Epoch 46/50
688/688             3s 4ms/step -
accuracy: 0.6740 - loss: 0.7124
Epoch 47/50
688/688             3s 4ms/step -
accuracy: 0.6757 - loss: 0.7188
Epoch 48/50
688/688             3s 4ms/step -
accuracy: 0.6731 - loss: 0.7210
Epoch 49/50
688/688             3s 4ms/step -
accuracy: 0.6733 - loss: 0.7229
Epoch 50/50
688/688             3s 4ms/step -
accuracy: 0.6742 - loss: 0.7202
63/63             0s 4ms/step
Model: Best Random Search
Training Time: 135.883
acc: 0.6555
prec:0.7839
recall:0.6555
```

Interesting! an extremely deep network is preferred. With a range of 100-200 per network. Batch normalization is preferred. Averaging is not required. We are using Batch Normalization and have a decay rate of around .97

Some notes after looking in Tensorboard: 1. Activation Function has no clear winner even after looking at ~10 best models. 2. Similarly averaging doesn't seem to have that big of a difference. The absolute best performed well without it though. 3. Batch norm performed best on the best models, but also had a fairly large split. 4. Higher decay rate seems (mostly preferred) 5. Dropout of zero is MOST LIKELY BEST. biggest indicator so far.

Everything else had a fairly large spread. Lets assume a lower dropout rate and try a different tuning method and see if we can gain more info. Since Selu Won last time we are going to try introducion a norm layer to see if that combo with selu and batch norm can get us somewhere.

this inital search was quite flawed. We respectfully ask you don't consider it when grading, it is being included so you can still observe thought process and progression.

Reasons it was flawed: 1. Selu was used without normalization layer (stupid mistake) 2. learning rate wasn't kept constant (MASSIVE MISTAKE). because of this a potentially great model could have performed poorly due to having too high a learning rate or too low a learning rate. Keeping it consistent is ideal, but even better is putting measures in place to make sure learning rate is less of a factor. 3. The inclusion of preprocessing screening may muddle the search for an ideal model. Averaging twice could drop performance of an otherwise good model by a lot. this muddles our search for truly well architectured model.

[28]:
```
len(x_tr) // 32
```

[28]:
```
687
```

[29]:
```
from utils import flatten_data

def build_tuned_wide_and_deep(hp, meta, mean, variance, seed=42):
    X_shape = meta["X_shape_"]
    n_features = X_shape[1]

    activation = hp.Choice("activation", values=["relu", "leaky_relu", "elu",␣
↪"selu"])
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    #learning_rate = hp.Choice("learning_rate", values=[3e-4, 1e-3])
    #optimizer = hp.Choice("optimizer", values=["sgd", "adam"]) #Just adam for␣
↪now
    batch_norm = hp.Boolean("batch_norm")
    #decay_rate = hp.Float("decay_rate", min_value=0.9, max_value=0.99, step=0.
↪01)
    wide_and_deep = hp.Boolean("wide_and_deep")
    if wide_and_deep:
        wide_frac = hp.Float("wide_frac", min_value=.7, max_value=1)
        deep_frac = hp.Float("deep_frac", min_value=.7, max_value=1)
        rng = np.random.RandomState(seed)
        wide_idx = rng.choice(n_features, size=int(n_features * wide_frac),␣
↪replace=False)
        deep_idx = rng.choice(n_features, size=int(n_features * deep_frac),␣
↪replace=False)
    else:
        deep_idx = np.arange(0, n_features)
    '''
    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(
            learning_rate=tf.keras.optimizers.schedules.ExponentialDecay(
                initial_learning_rate=learning_rate,
                decay_steps=100,
                decay_rate=decay_rate,
```

```
            )
        )
    '''
    lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
        initial_learning_rate=3e-3,
        decay_steps=687 * 50,
        warmup_target=3e-3,
        warmup_steps=687 * 2,  # 2 epochs of warmup
    )

    optimizer = tf.keras.optimizers.Adam(learning_rate=lr_schedule)

    inputs = tf.keras.layers.Input(shape=(n_features,))

    if activation == "selu":
        norm_layer = tf.keras.layers.Normalization(mean=mean, variance=variance)
        preprocessing = norm_layer(inputs)
    else:
        preprocessing = tf.keras.layers.Rescaling(1./255)(inputs)

    if wide_and_deep:
        wide_input = tf.keras.layers.Lambda(lambda x: tf.gather(x, wide_idx,␣
↪axis=1))(preprocessing)

    deep_input = tf.keras.layers.Lambda(lambda x: tf.gather(x, deep_idx,␣
↪axis=1))(preprocessing)

    last_layer = deep_input
    for i in range(n_hidden):
        n_neurons = hp.Int(f"n_neurons{i}", min_value=16, max_value=256)
        last_layer = tf.keras.layers.Dense(n_neurons,␣
↪activation=activation)(last_layer)
        if batch_norm:
            last_layer = tf.keras.layers.BatchNormalization()(last_layer)

    if wide_and_deep:
        last_layer = tf.keras.layers.Concatenate()([last_layer, wide_input])

    outputs = tf.keras.layers.Dense(4, activation="softmax")(last_layer)
    model = tf.keras.Model(inputs=inputs, outputs=outputs)

    model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,␣
↪metrics=["accuracy"])
    return model

class MyTuningModel(kt.HyperModel):
    def __init__(self, meta, x_tr):
```

```
        super().__init__()
        self.meta = meta
        norm_layer = tf.keras.layers.Normalization()
        norm_layer.adapt(x_tr)
        self.mean_, self.var_ = norm_layer.get_weights()[0], norm_layer.
↪get_weights()[1] #getting our mean and var incase we use selu


    def fit(self, hp, model, X, y, **kwargs):
        '''
        if hp.Boolean("median"):
            X = median(X)
        if hp.Boolean("avg"):
            X = avg_2x2_pool(X)

        '''

        X = flatten_data(X)

        return model.fit(X, y, **kwargs)

    def build(self, hp):
        return build_tuned_wide_and_deep(hp, self.meta, mean=self.mean_,␣
↪variance=self.var_)
```

To briefly explain my rationale. My goal here is to build an overal architecture that works well. Things like number of layers, whether or not to use wide and deep, number of nodes per layer, activation function, things that have a wide sweeping impact on the overall model is what we are titrating for right now. Because of this some ideas that I was originally going to include have been commented out as they may muddy my original objective. I experienced a little bit of "Search Creep" when making this where I just kept adding more and more to the search and felt like my net was cast too wide, so we have cut back to what we need. Maybe most critical difference from the first search: The learning rate is being constrained. This parameter controls too much of performance so it was opted to be constrained. Sample weights are also being introduced as we want to make sure the model generalizes well to all classes, a decision that in hindsight should've been made long ago

[30]:
```
run_logdir = get_run_logdir('my_logs/architecture_search')

tensorboard_cb =  tf.keras.callbacks.TensorBoard(run_logdir)
#profile batching is removed (the callback was taken from the textbook)
#profile batching just collects data on general performance metrics
#such as how long something takes on the CPU
#or which layer is the slowests
#since our hyperparameters are changing constantly this info isn't very useful.
```

It's time to go into more detail on Tuner Selection. Looking at documentation there is really 4

choices: 1. RandomSearch 2. GridSearch 3. BayesianOptimization 4. Hyperband

GridSearch is by far the easiest to rule out we are searching over so many parameters it would just be impossible to scan over all of them. Especially when we are dealing with continuous variables. Random Search vs Hyperband is a tougher choice. Hyperband could essentially be viewed as a 'smarter' Random Search. However this search will be done for a WHILE. and because of this we are able to be far more forgiving on what Hyperband tends to explore while still maintaining the cost cutting benefits of it. So in actuality this should lead to just more raw models being explored. Finally we have Bayesian optimization: The problem here is in structure versus execution. Bayesian optimization assumes a function: f(hyperparemeters) -> Loss/Val_accuracy/accuracy that we can model. Essentially something like given these datapoints probability that our function looks like this is. and then probes new points smartly based around where in the feature space we know is good and where is unknown. The problem with this is the inital assumption. Bayesian optimization assumes that if you have mostly similar hyperparameters you will get mostly a similar results. And this is simply not the case with our design, because we have Booleans. If we used a boolean for wide and deep for example and it was true and had a deep layer of 2, we might wrongly explore that the feature space with deep layers of 2 for action space of two and generalize this architectures where it doesn't belong like a non wide and deep model. The reality is our model is certianly not continuous in nature. something as simple as chaning wide & deep can mean massive changes for what is optimal for the other hyperparameters, the feature space is likely jagged and rough. This is an exaggeration of the problem, in practice BayesianOptimization would probably still work well, but perhaps not the best.

[31]:
```
x_tr_flat = flatten_data(x_tr)
x_va_flat = flatten_data(x_va)
#flattening so preprocessing runs smoothly

meta = {"X_shape_": (x_tr_flat.shape[0], x_tr_flat.shape[1])} #Necessary to␣
 ↪give the model and idea of input shape
#already preflatted so logic stayes the same

print("x_tr_flat shape:", x_tr_flat.shape)
print("x_va_flat shape:", x_va_flat.shape)
print("meta:", meta)
print("sample_weights shape:", sample_weights.shape)
print("sample_weights_valid shape:", sample_weights_valid.shape)
```

```
x_tr_flat shape: (22000, 784)
x_va_flat shape: (2000, 784)
meta: {'X_shape_': (22000, 784)}
sample_weights shape: (22000,)
sample_weights_valid shape: (2000,)
```

The idea behind this is that: A poor architecture model will still perform a good architecture model with a learning rate that causes diveragence or too low training. Our solution to this is twofold: One we will use a learning_rate schedule. A learning rate schedule modifies the learning rate as training goes on to solve this very problem. We will use Cosine Decay with a warm up. We will start out with a higher learning rate so slower models can catch up quickly then quickly peel back as we aproach optimal solutions. Secondly we will use the above callback. if training is

not improving from the aggressive start it will begin lowering the learning rate in hopes that it will peel back in time.

I plan on letting this run overnight so we'll see the results in the morning. I am not too concerned with training time because of this.

[32]:
```
tuner = kt.Hyperband(
    hypermodel=MyTuningModel(meta=meta, x_tr=x_tr_flat),
    objective="val_accuracy",
    max_epochs=75,
    factor=2,
    hyperband_iterations=3,
    directory="./hyperband_dir",
    project_name="Architecture_tuning",
    overwrite=False,
    seed=42
    )

'''
tuner.search(
    x_tr_flat,
    y_tr,
    validation_data=(x_va_flat, y_va, sample_weights_valid),
    sample_weight=sample_weights,
    batch_size=32,
    callbacks=[early_stopping_cb, tensorboard_cb],
    verbose=1
)
'''
```

```
Reloading Tuner from ./hyperband_dir\Architecture_tuning\tuner0.json
```

[32]:
```
'\ntuner.search(\n    x_tr_flat, \n    y_tr,\n    validation_data=(x_va_flat,
y_va, sample_weights_valid),\n    sample_weight=sample_weights,\n
batch_size=32,\n    callbacks=[early_stopping_cb, tensorboard_cb],\n
verbose=1\n)\n'
```

1076 Different Neural Network Architectures built and tested lets see the results. I am running into a very annoying problem. I now have too much data for my laptop to open with Tensorboard. My laptop is literally crashing as I try to open it. And I can't upload it to the github as that is too intensive for my laptop as well.

[33]:
```
best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]
best_model_hyper = tuner.hypermodel.build(best_hp)

best_model_hyper.summary()
```

```
Model: "functional_10"
```

Layer (type)	Output Shape	Param #
input_layer_10 (InputLayer)	(None, 784)	0
normalization_1 (Normalization)	(None, 784)	0
lambda_2 (Lambda)	(None, 784)	0
dense_34 (Dense)	(None, 184)	144,440
batch_normalization_8 (BatchNormalization)	(None, 184)	736
dense_35 (Dense)	(None, 206)	38,110
batch_normalization_9 (BatchNormalization)	(None, 206)	824
dense_36 (Dense)	(None, 192)	39,744
batch_normalization_10 (BatchNormalization)	(None, 192)	768
dense_37 (Dense)	(None, 120)	23,160
batch_normalization_11 (BatchNormalization)	(None, 120)	480
dense_38 (Dense)	(None, 138)	16,698
batch_normalization_12 (BatchNormalization)	(None, 138)	552
dense_39 (Dense)	(None, 4)	556

Total params: 266,068 (1.01 MB)

Trainable params: 264,388 (1.01 MB)

Non-trainable params: 1,680 (6.56 KB)

[34]:
```
results.append(train_eval_no_wrapper(
                model=best_model_hyper,
```

```
                name="Best Hyperband Result",
                x_tr=x_tr_flat,
                y_tr=y_tr,
                x_va=x_va_flat,
                y_va=y_va,
                sample_weights=sample_weights,
                callbacks=[early_stopping_cb]))
```

```
Epoch 1/50
688/688             5s 4ms/step -
accuracy: 0.4300 - loss: 1.2905
Epoch 2/50
 41/688             2s 4ms/step -
accuracy: 0.5151 - loss: 1.1424
```

```
c:\Users\samue\anaconda3\Lib\site-
packages\keras\src\callbacks\early_stopping.py:99: UserWarning: Early stopping
conditioned on metric `val_loss` which is not available. Available metrics are:
accuracy,loss
  current = self.get_monitor_value(logs)
```

```
688/688             3s 4ms/step -
accuracy: 0.5052 - loss: 1.1554
Epoch 3/50
688/688             3s 4ms/step -
accuracy: 0.5413 - loss: 1.0860
Epoch 4/50
688/688             3s 4ms/step -
accuracy: 0.5732 - loss: 1.0186
Epoch 5/50
688/688             3s 4ms/step -
accuracy: 0.5901 - loss: 0.9735
Epoch 6/50
688/688             3s 4ms/step -
accuracy: 0.6099 - loss: 0.9234
Epoch 7/50
688/688             3s 4ms/step -
accuracy: 0.6196 - loss: 0.8983
Epoch 8/50
688/688             3s 4ms/step -
accuracy: 0.6362 - loss: 0.8761
Epoch 9/50
688/688             2s 4ms/step -
accuracy: 0.6428 - loss: 0.8390
Epoch 10/50
688/688             3s 4ms/step -
accuracy: 0.6515 - loss: 0.8196
Epoch 11/50
688/688             3s 4ms/step -
```

```
accuracy: 0.6566 - loss: 0.7954
Epoch 12/50
688/688             3s 4ms/step -
accuracy: 0.6661 - loss: 0.7724
Epoch 13/50
688/688             2s 4ms/step -
accuracy: 0.6760 - loss: 0.7384
Epoch 14/50
688/688             3s 4ms/step -
accuracy: 0.6816 - loss: 0.7227
Epoch 15/50
688/688             3s 4ms/step -
accuracy: 0.6940 - loss: 0.6962
Epoch 16/50
688/688             2s 4ms/step -
accuracy: 0.7044 - loss: 0.6769
Epoch 17/50
688/688             2s 3ms/step -
accuracy: 0.7085 - loss: 0.6595
Epoch 18/50
688/688             2s 3ms/step -
accuracy: 0.7161 - loss: 0.6411
Epoch 19/50
688/688             2s 3ms/step -
accuracy: 0.7307 - loss: 0.6099
Epoch 20/50
688/688             2s 3ms/step -
accuracy: 0.7371 - loss: 0.5922
Epoch 21/50
688/688             2s 3ms/step -
accuracy: 0.7425 - loss: 0.5824
Epoch 22/50
688/688             2s 3ms/step -
accuracy: 0.7516 - loss: 0.5548
Epoch 23/50
688/688             2s 3ms/step -
accuracy: 0.7546 - loss: 0.5406
Epoch 24/50
688/688             2s 4ms/step -
accuracy: 0.7720 - loss: 0.5096
Epoch 25/50
688/688             3s 4ms/step -
accuracy: 0.7742 - loss: 0.4979
Epoch 26/50
688/688             36s 52ms/step -
accuracy: 0.7816 - loss: 0.4818
Epoch 27/50
688/688             3s 5ms/step -
```

```
accuracy: 0.7871 - loss: 0.4648
Epoch 28/50
688/688             3s 5ms/step -
accuracy: 0.7970 - loss: 0.4440
Epoch 29/50
688/688             3s 4ms/step -
accuracy: 0.8004 - loss: 0.4378
Epoch 30/50
688/688             3s 4ms/step -
accuracy: 0.8087 - loss: 0.4140
Epoch 31/50
688/688             3s 4ms/step -
accuracy: 0.8147 - loss: 0.4034
Epoch 32/50
688/688             3s 4ms/step -
accuracy: 0.8180 - loss: 0.3904
Epoch 33/50
688/688             3s 4ms/step -
accuracy: 0.8283 - loss: 0.3719
Epoch 34/50
688/688             3s 4ms/step -
accuracy: 0.8294 - loss: 0.3568
Epoch 35/50
688/688             3s 4ms/step -
accuracy: 0.8350 - loss: 0.3450
Epoch 36/50
688/688             3s 4ms/step -
accuracy: 0.8428 - loss: 0.3315
Epoch 37/50
688/688             3s 4ms/step -
accuracy: 0.8471 - loss: 0.3202
Epoch 38/50
688/688             3s 4ms/step -
accuracy: 0.8573 - loss: 0.3014
Epoch 39/50
688/688             3s 4ms/step -
accuracy: 0.8577 - loss: 0.2942
Epoch 40/50
688/688             3s 4ms/step -
accuracy: 0.8604 - loss: 0.2886
Epoch 41/50
688/688             3s 4ms/step -
accuracy: 0.8709 - loss: 0.2703
Epoch 42/50
688/688             3s 4ms/step -
accuracy: 0.8717 - loss: 0.2621
Epoch 43/50
688/688             3s 4ms/step -
```

```
accuracy: 0.8762 - loss: 0.2555
Epoch 44/50
688/688             3s 4ms/step -
accuracy: 0.8786 - loss: 0.2489
Epoch 45/50
688/688             3s 4ms/step -
accuracy: 0.8854 - loss: 0.2378
Epoch 46/50
688/688             3s 4ms/step -
accuracy: 0.8845 - loss: 0.2373
Epoch 47/50
688/688             3s 4ms/step -
accuracy: 0.8877 - loss: 0.2300
Epoch 48/50
688/688             3s 4ms/step -
accuracy: 0.8885 - loss: 0.2325
Epoch 49/50
688/688             3s 4ms/step -
accuracy: 0.8871 - loss: 0.2262
Epoch 50/50
688/688             3s 4ms/step -
accuracy: 0.8885 - loss: 0.2309
63/63             0s 3ms/step
Model: Best Hyperband Result
Training Time: 166.163
acc: 0.7680
prec:0.7935
recall:0.7680
```

Had to reload as my laptop crashed trying to access tensorboard.

[35]:
```python
import matplotlib.pyplot as plt
import pandas as pd


trials = tuner.oracle.get_best_trials(num_trials=100)
rows = []
for trial in trials:
    row = {"trial_id": trial.trial_id, "val_accuracy": trial.score}
    row.update(trial.hyperparameters.values)
    rows.append(row)
df = pd.DataFrame(rows).sort_values("val_accuracy", ascending=False).
 ↪reset_index(drop=True)


# Figure 1: top 20 trials as a table
top20 = df.head(20).copy()
```

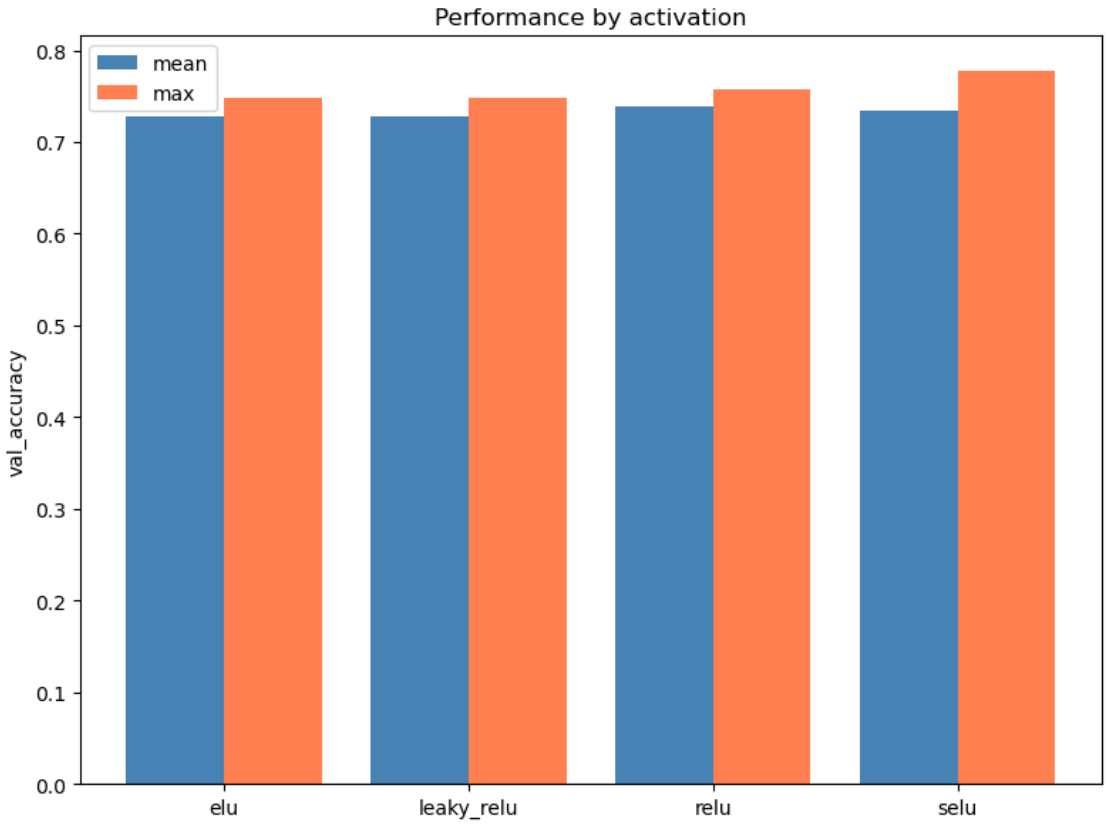
```
# Drop columns that aren't useful in the display (keep if you want them)
cols_to_show = [c for c in top20.columns if c not in {"tuner/trial_id", "tuner/
 ↪epochs", "tuner/initial_epoch", "tuner/bracket", "tuner/round"}]
top20_display = top20[cols_to_show]

# Round floats for readability
for col in top20_display.select_dtypes(include="float").columns:
    top20_display[col] = top20_display[col].round(4)

fig, ax = plt.subplots(figsize=(max(12, len(cols_to_show) * 1.2),␣
 ↪len(top20_display) * 0.4 + 1))
ax.axis("off")
table = ax.table(
    cellText=top20_display.values,
    colLabels=top20_display.columns,
    cellLoc="center",
    loc="center",
)
table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.4)

# Bold the header row
for j in range(len(top20_display.columns)):
    cell = table[0, j]
    cell.set_text_props(weight="bold")
    cell.set_facecolor("#404040")
    cell.set_text_props(color="white")

# Highlight the best row
for j in range(len(top20_display.columns)):
    table[1, j].set_facecolor("#d4edda")

ax.set_title("Top 20 trials by val_accuracy", pad=20, fontsize=12,␣
 ↪weight="bold")
plt.tight_layout()
plt.savefig("top20_trials.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```

Top 20 trials by val_accuracy

trial_id	val_accuracy	activation	n_hidden	batch_norm	wide_and_deep	n_neurons0	n_neurons1	n_neurons2	n_neurons3	wide_frac	deep_frac	n_neurons4	n_neurons5	n_neurons6	n_neurons7
0235	0.777	selu	5	True	False	184	206	192	120	0.8165	0.8641	138	49	108	37
0318	0.7695	selu	3	True	False	145	244	152	240	0.9111	0.7363	31	191	137	139
1054	0.7675	selu	2	True	False	223	66	209	235	0.7905	0.7051	188	213	115	52
0866	0.7625	selu	4	True	False	175	191	230	171	0.9491	0.9508	217	54	127	173
0647	0.7575	relu	4	True	True	205	96	228	128	0.898	0.8692	115	88	102	213
1049	0.7525	selu	2	True	False	223	66	209	235	0.7905	0.7051	188	213	115	52
0505	0.752	selu	2	False	False	58	238	65	138	0.7746	0.82	220	65	45	217
0317	0.75	selu	4	False	False	229	149	142	87	0.9711	0.7384	159	213	95	220
0951	0.75	selu	2	False	False	26	120	78	121	0.9896	0.8337	34	254	205	226
0315	0.749	selu	4	False	False	229	149	142	87	0.9711	0.7384	159	213	95	220
0676	0.7485	leaky_relu	2	True	False	223	87	207	98	0.8468	0.7219	112	251	181	172
1065	0.7485	relu	5	True	True	201	176	191	19	0.8867	0.9573	221	24	175	120
0865	0.7475	selu	4	True	False	175	191	230	171	0.9491	0.9508	217	54	127	173
0331	0.7475	elu	2	False	False	255	189	245	111	0.8047	0.7424	124	200	92	57
0867	0.747	selu	3	False	False	134	252	21	163	0.9498	0.9173	139	165	241	88
0683	0.746	selu	2	False	False	154	175	173	67	0.8991	0.9162	122	242	120	205
0149	0.7445	selu	3	True	False	113	45	58	104	0.927	0.7927	148	205	115	83
0706	0.744	leaky_relu	4	False	True	83	66	110	208	0.9043	0.9896	81	18	217	117
1036	0.744	selu	8	True	False	137	18	250	147	0.8101	0.8874	88	116	31	183
0689	0.7435	selu	2	False	False	154	175	173	67	0.8991	0.9162	122	242	120	205

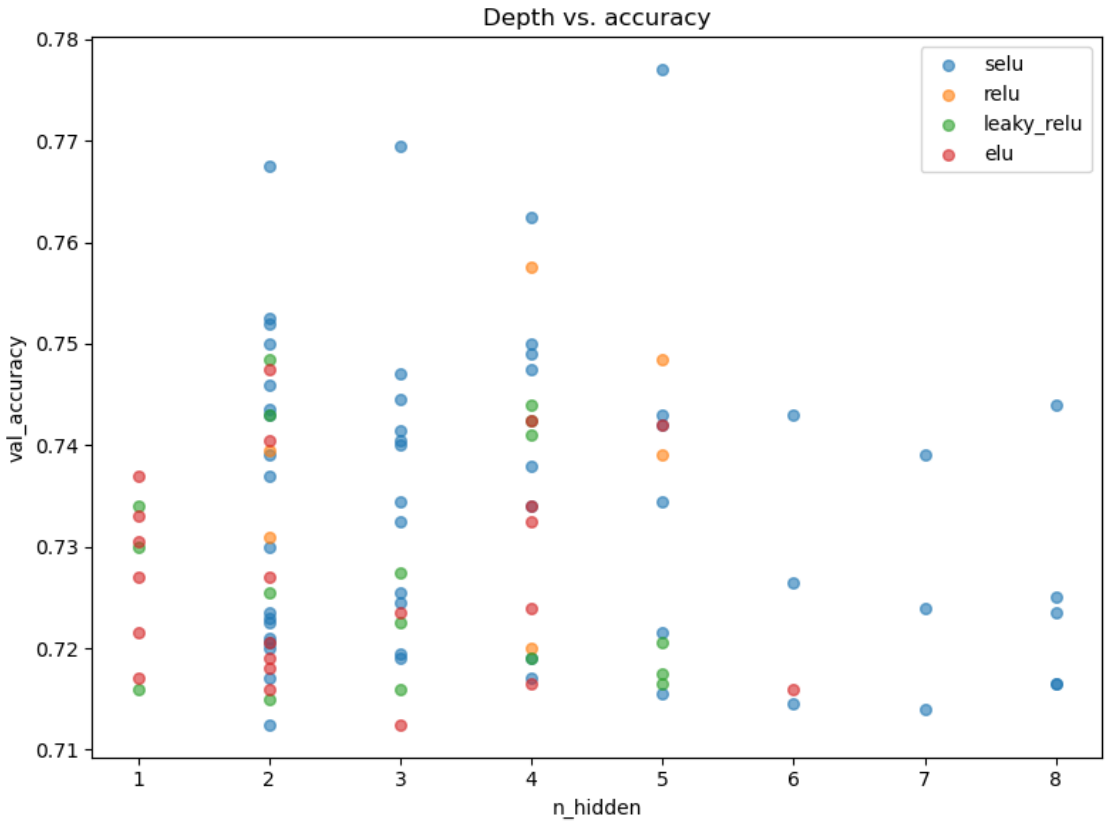
Some notes here: Of the activations Selu is definitely the majority. n_hidden varies quite a lot. seems to mostly be constrainted in the 2-4 range though. (shallower networks). Batch norm has quite the mix. This makes sense as Selu is a replacement for BatchNorm. Wide & Deep is majority False. Neuron count should maybe have been held constant across layers. Hard to interperet, however there is often a slight dip in neuron count in the models last layer.

[36]:
```
fig, ax = plt.subplots(figsize=(8, 6))
activation_stats = df.groupby("activation")["val_accuracy"].agg(["mean", "max"])
x = range(len(activation_stats))
ax.bar([i - 0.2 for i in x], activation_stats["mean"], width=0.4, label="mean",␣
 ↪color="steelblue")
ax.bar([i + 0.2 for i in x], activation_stats["max"], width=0.4, label="max",␣
 ↪color="coral")
ax.set_xticks(x)
ax.set_xticklabels(activation_stats.index)
ax.set_ylabel("val_accuracy")
ax.set_title("Performance by activation")
ax.legend()
plt.tight_layout()
plt.savefig("activation_performance.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```



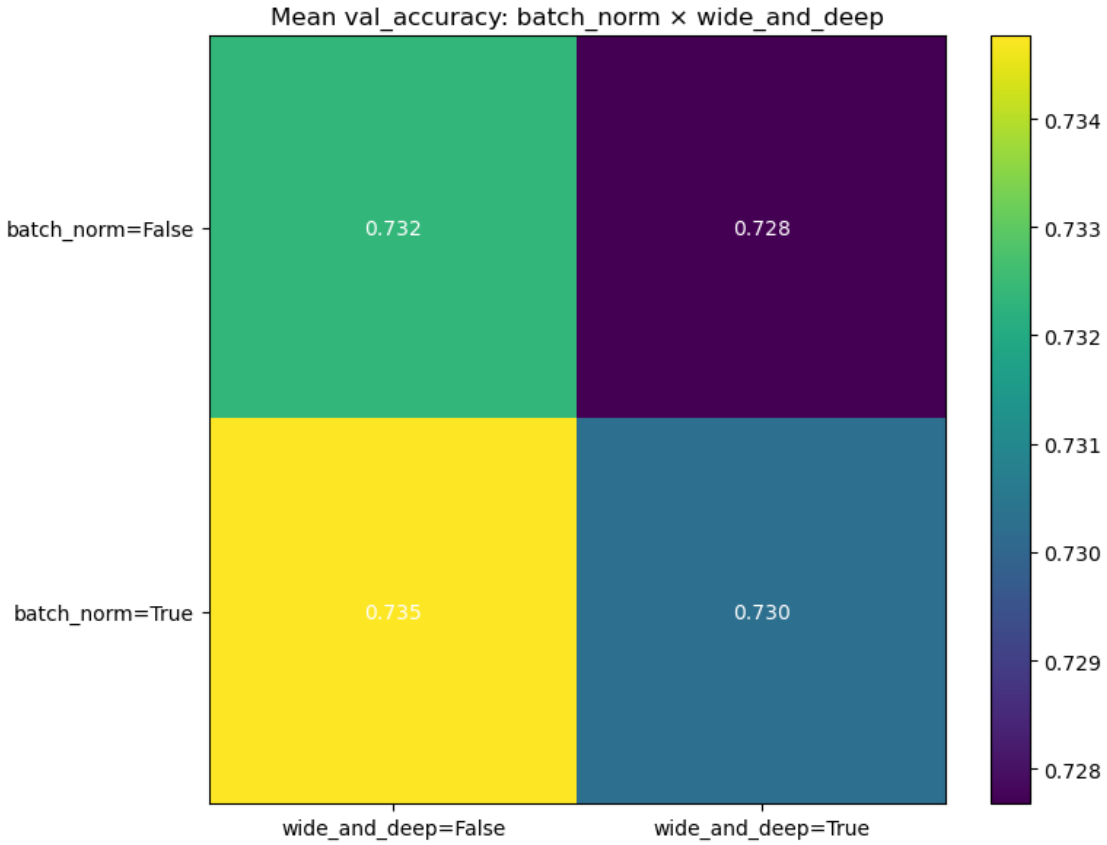
While their averages are roungly the same selu has a much higher peak.

[37]:
```
fig, ax = plt.subplots(figsize=(8, 6))
for activation in df["activation"].unique():
    subset = df[df["activation"] == activation]
    ax.scatter(subset["n_hidden"], subset["val_accuracy"],
               label=activation, alpha=0.6, s=30)
ax.set_xlabel("n_hidden")
ax.set_ylabel("val_accuracy")
ax.set_title("Depth vs. accuracy")
ax.legend()
plt.tight_layout()
plt.savefig("depth_vs_accuracy.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```



Generally Selu is performing the best once again. specifically on deeper models. too. the best accuracy for each layer except 1 is Selu.

[38]:
```
fig, ax = plt.subplots(figsize=(8, 6))
pivot = df.pivot_table(values="val_accuracy", index="batch_norm",
                       columns="wide_and_deep", aggfunc="mean")
im = ax.imshow(pivot.values, cmap="viridis", aspect="auto")
ax.set_xticks(range(len(pivot.columns)))
ax.set_xticklabels([f"wide_and_deep={c}" for c in pivot.columns])
ax.set_yticks(range(len(pivot.index)))
ax.set_yticklabels([f"batch_norm={i}" for i in pivot.index])
ax.set_title("Mean val_accuracy: batch_norm × wide_and_deep")
for i in range(pivot.shape[0]):
    for j in range(pivot.shape[1]):
        ax.text(j, i, f"{pivot.values[i, j]:.3f}",
                ha="center", va="center", color="white")
plt.colorbar(im, ax=ax)
plt.tight_layout()
plt.savefig("batchnorm_widedeep_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()
plt.close()
```



Models on average perform best when batch norm is true and wide and deep is false. Theory Time: Wide and deep is not helping input too much. especially when subsetting. Batch normalization or Selu is a substitution seems almost essential for the model though.

A worry though: selu could be advantaged because of normalized inputs and not because of actual improvement. To see lets build a relu

[39]:
```
def build_test_relu(meta, mean, variance, seed=42):
    X_shape = meta["X_shape_"]
    n_features = X_shape[1]

    activation = "relu"
    n_hidden = 5
    #learning_rate = hp.Choice("learning_rate", values=[3e-4, 1e-3])
    #optimizer = hp.Choice("optimizer", values=["sgd", "adam"]) #Just adam for now
    batch_norm = True
    #decay_rate = hp.Float("decay_rate", min_value=0.9, max_value=0.99, step=0.01)
    wide_and_deep = False
    '''
```

```
    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(
            learning_rate=tf.keras.optimizers.schedules.ExponentialDecay(
                initial_learning_rate=learning_rate,
                decay_steps=100,
                decay_rate=decay_rate,
            )
        )
    '''
    lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
        initial_learning_rate=3e-3,
        decay_steps=687 * 50,
        warmup_target=3e-3,
        warmup_steps=687 * 2,  # 2 epochs of warmup
    )

    optimizer = tf.keras.optimizers.Adam(learning_rate=lr_schedule)

    inputs = tf.keras.layers.Input(shape=(n_features,))

    norm_layer = tf.keras.layers.Normalization(mean=mean, variance=variance)
    preprocessing = norm_layer(inputs)


    last_layer = preprocessing
    for i in range(n_hidden):
        n_neurons = 200
        last_layer = tf.keras.layers.Dense(n_neurons,␣
↪activation=activation)(last_layer)
        if batch_norm:
            last_layer = tf.keras.layers.BatchNormalization()(last_layer)

    outputs = tf.keras.layers.Dense(4, activation="softmax")(last_layer)
    model = tf.keras.Model(inputs=inputs, outputs=outputs)

    model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,␣
↪metrics=["accuracy"])
    return model

norm_layer = tf.keras.layers.Normalization()
norm_layer.adapt(x_tr_flat)
mean_, var_ = norm_layer.get_weights()[0], norm_layer.get_weights()[1]

model_relu = build_test_relu(meta=meta, mean=mean_, variance=var_)
```

[40]:
```
results.append(train_eval_no_wrapper(
    model=model_relu,
```

```
    name="ReLu with best Hyperparams",
    x_tr=x_tr_flat,
    y_tr=y_tr,
    x_va=x_va_flat,
    y_va=y_va,
    sample_weights=sample_weights,
    callbacks=[early_stopping_cb]
))
```

```
Epoch 1/50
688/688             5s 4ms/step -
accuracy: 0.4257 - loss: 1.3242
Epoch 2/50
 45/688             2s 3ms/step -
accuracy: 0.4836 - loss: 1.2392
```

```
c:\Users\samue\anaconda3\Lib\site-
packages\keras\src\callbacks\early_stopping.py:99: UserWarning: Early stopping
conditioned on metric `val_loss` which is not available. Available metrics are:
accuracy,loss
  current = self.get_monitor_value(logs)
```

```
688/688             3s 4ms/step -
accuracy: 0.5141 - loss: 1.1584
Epoch 3/50
688/688             3s 4ms/step -
accuracy: 0.5440 - loss: 1.0735
Epoch 4/50
688/688             3s 4ms/step -
accuracy: 0.5774 - loss: 1.0193
Epoch 5/50
688/688             3s 4ms/step -
accuracy: 0.5923 - loss: 0.9754
Epoch 6/50
688/688             3s 4ms/step -
accuracy: 0.6006 - loss: 0.9474
Epoch 7/50
688/688             3s 4ms/step -
accuracy: 0.6171 - loss: 0.9135
Epoch 8/50
688/688             3s 4ms/step -
accuracy: 0.6187 - loss: 0.8821
Epoch 9/50
688/688             3s 4ms/step -
accuracy: 0.6355 - loss: 0.8587
Epoch 10/50
688/688             3s 4ms/step -
accuracy: 0.6428 - loss: 0.8373
```

```
Epoch 11/50
688/688             3s 4ms/step -
accuracy: 0.6557 - loss: 0.8035
Epoch 12/50
688/688             3s 4ms/step -
accuracy: 0.6676 - loss: 0.7833
Epoch 13/50
688/688             2s 4ms/step -
accuracy: 0.6749 - loss: 0.7598
Epoch 14/50
688/688             3s 4ms/step -
accuracy: 0.6788 - loss: 0.7438
Epoch 15/50
688/688             3s 5ms/step -
accuracy: 0.6960 - loss: 0.7185
Epoch 16/50
688/688             4s 5ms/step -
accuracy: 0.6969 - loss: 0.6915
Epoch 17/50
688/688             3s 4ms/step -
accuracy: 0.7079 - loss: 0.6752
Epoch 18/50
688/688             3s 4ms/step -
accuracy: 0.7116 - loss: 0.6553
Epoch 19/50
688/688             3s 4ms/step -
accuracy: 0.7257 - loss: 0.6259
Epoch 20/50
688/688             2s 3ms/step -
accuracy: 0.7314 - loss: 0.6074
Epoch 21/50
688/688             2s 3ms/step -
accuracy: 0.7375 - loss: 0.5953
Epoch 22/50
688/688             3s 4ms/step -
accuracy: 0.7483 - loss: 0.5710
Epoch 23/50
688/688             2s 3ms/step -
accuracy: 0.7574 - loss: 0.5507
Epoch 24/50
688/688             2s 3ms/step -
accuracy: 0.7699 - loss: 0.5192
Epoch 25/50
688/688             3s 4ms/step -
accuracy: 0.7721 - loss: 0.5035
Epoch 26/50
688/688             2s 3ms/step -
accuracy: 0.7781 - loss: 0.4844
```

```
Epoch 27/50
688/688             2s 3ms/step -
accuracy: 0.7807 - loss: 0.4753
Epoch 28/50
688/688             2s 3ms/step -
accuracy: 0.7948 - loss: 0.4486
Epoch 29/50
688/688             2s 3ms/step -
accuracy: 0.7999 - loss: 0.4298
Epoch 30/50
688/688             2s 3ms/step -
accuracy: 0.8105 - loss: 0.4189
Epoch 31/50
688/688             2s 3ms/step -
accuracy: 0.8205 - loss: 0.3962
Epoch 32/50
688/688             2s 3ms/step -
accuracy: 0.8287 - loss: 0.3726
Epoch 33/50
688/688             2s 3ms/step -
accuracy: 0.8320 - loss: 0.3622
Epoch 34/50
688/688             2s 3ms/step -
accuracy: 0.8370 - loss: 0.3507
Epoch 35/50
688/688             2s 3ms/step -
accuracy: 0.8468 - loss: 0.3253
Epoch 36/50
688/688             2s 3ms/step -
accuracy: 0.8513 - loss: 0.3125
Epoch 37/50
688/688             2s 3ms/step -
accuracy: 0.8605 - loss: 0.3030
Epoch 38/50
688/688             2s 3ms/step -
accuracy: 0.8647 - loss: 0.2878
Epoch 39/50
688/688             2s 3ms/step -
accuracy: 0.8670 - loss: 0.2722
Epoch 40/50
688/688             2s 3ms/step -
accuracy: 0.8753 - loss: 0.2596
Epoch 41/50
688/688             2s 3ms/step -
accuracy: 0.8756 - loss: 0.2614
Epoch 42/50
688/688             2s 3ms/step -
accuracy: 0.8830 - loss: 0.2391
```

```
Epoch 43/50
688/688             2s 3ms/step -
accuracy: 0.8876 - loss: 0.2369
Epoch 44/50
688/688             2s 3ms/step -
accuracy: 0.8875 - loss: 0.2374
Epoch 45/50
688/688             2s 3ms/step -
accuracy: 0.8925 - loss: 0.2231
Epoch 46/50
688/688             2s 4ms/step -
accuracy: 0.8932 - loss: 0.2200
Epoch 47/50
688/688             2s 3ms/step -
accuracy: 0.9007 - loss: 0.2066
Epoch 48/50
688/688             2s 3ms/step -
accuracy: 0.9015 - loss: 0.2103
Epoch 49/50
688/688             2s 3ms/step -
accuracy: 0.9010 - loss: 0.2044
Epoch 50/50
688/688             2s 3ms/step -
accuracy: 0.8985 - loss: 0.2075
63/63             0s 3ms/step
Model: ReLu with best Hyperparams
Training Time: 123.681
acc: 0.7770
prec:0.7950
recall:0.7770
```

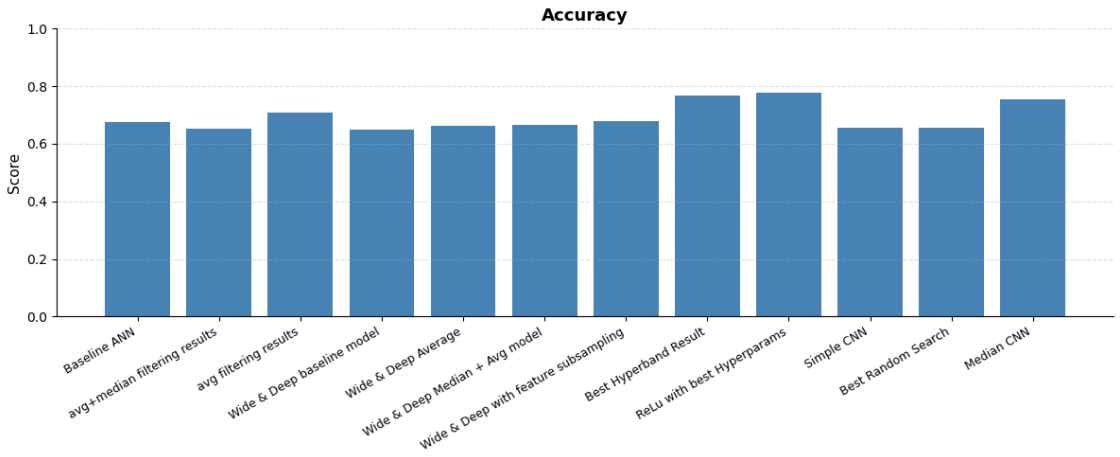
Val Accuracy is siginificantly lower. for this model than our best from the search. This suggest that SELU could actually be viable.

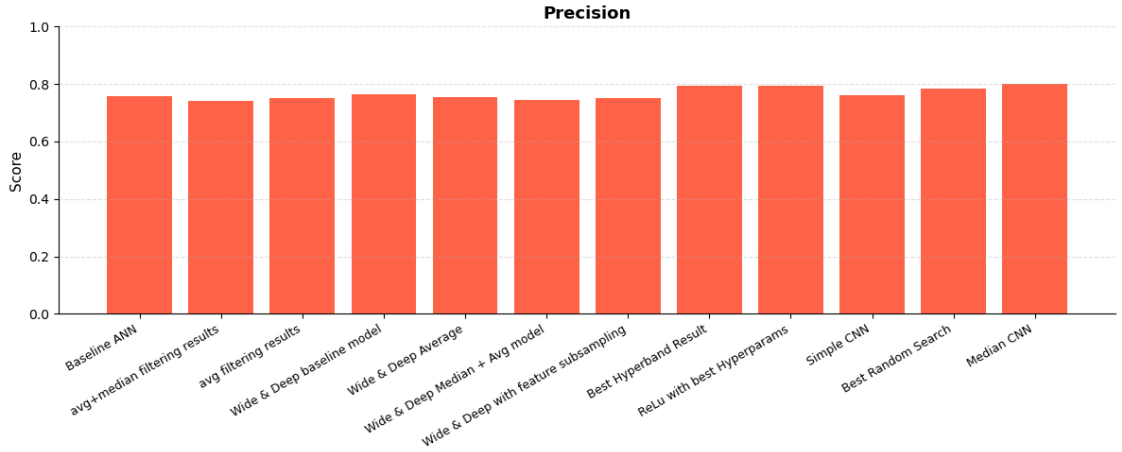
[88]:
```
model_names = [r["name"]   for r in results]
acc_scores  = [r["acc"]    for r in results]
prec_scores = [r["prec"]   for r in results]
rec_scores  = [r["recall"] for r in results]
f1_scores   = [r["f1"]     for r in results]

model_names
```

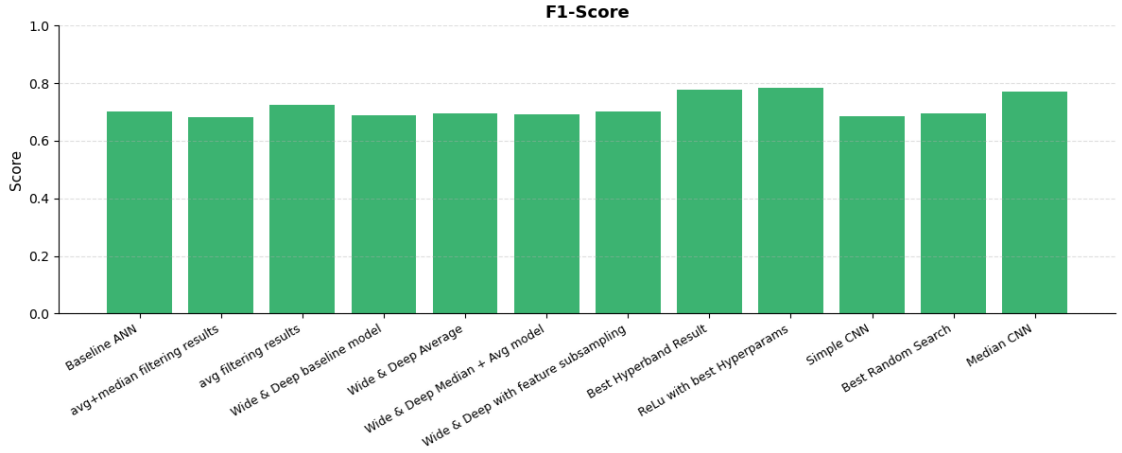
[88]:
```
['Baseline ANN',
 'avg+median filtering results',
 'avg filtering results',
 'Wide & Deep baseline model',
 'Wide & Deep Average',
```

```
'Wide & Deep Median + Avg model',
'Wide & Deep with feature subsampling',
'Best Hyperband Result',
'ReLu with best Hyperparams',
'Simple CNN',
'Best Random Search',
'Median CNN']
```

[92]:
```
metrics = [
    ("Accuracy",  acc_scores,  'steelblue'),
    ("Precision", prec_scores, 'tomato'),
    ("F1-Score",  f1_scores,   'mediumseagreen'),
]

for metric_name, scores, color in metrics:
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.bar(np.arange(len(model_names)), scores, color=color)

    ax.set_title(metric_name, fontsize=13, fontweight='bold')
    ax.set_ylabel("Score", fontsize=11)
    ax.set_xticks(np.arange(len(model_names)))
    ax.set_xticklabels(model_names, rotation=30, ha="right", fontsize=9)
    ax.set_ylim(0, 1)
    ax.grid(axis='y', linestyle='--', alpha=0.4)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    plt.tight_layout()
    plt.savefig(f'Figures/{metric_name}_All_Models')
    plt.show()
```




Precision
Score
1.0
0.8
0.6
0.4
0.2
0.0
Baseline ANN
avg+median filtering results
avg filtering results
Wide & Deep baseline model
Wide & Deep Average
Wide & Deep Median + Avg model
Wide & Deep with feature subsampling
Best Hyperband Result
ReLu with best Hyperparams
Simple CNN
Best Random Search
Median CNN


F1-Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
Baseline ANN
avg+median filtering results
avg filtering results
Wide & Deep baseline model
Wide & Deep Average
Wide & Deep Median + Avg model
Wide & Deep with feature subsampling
Best Hyperband Result
ReLu with best Hyperparams
Simple CNN
Best Random Search
Median CNN